

COMP H2704 – Web Application Security

TU757 / TU863

CA3 Project (50%)

Technical Report

Group: 32

Members:

Róisín Aylward – B00168689

Ingrid Francis – B00176783

Lecturer: Dr Vanessa Ayala-Rivera

Contents

School of Informatics & Cybersecurity - Assessment Submission Coversheet 3
School of Informatics & Cybersecurity - Assessment Submission Coversheet 4
Total Word Count: 4741 (excluding cover pages, references, figures, and appendices)..... 5
Completed Challenges 5
Individual Contributions: 6
Vulnerability Analyses 7
Vulnerability Analysis 1: Admin Section (Broken Access Control) – Ingrid Francis 7
Vulnerability Analysis 2: View Basket (Broken Access Control – IDOR) – Ingrid Francis 10
Vulnerability Analysis 3: SQL Injection – Róisín Aylward 12
Vulnerability Analysis 4: Broken Authentication – Róisín Aylward 16
Static Application Security Testing (SAST) 20
Introduction – Róisín Aylward 20
Methodology – Róisín Aylward 20
Environment Setup 20
Running the Scan 20
Scan Statistics 21
Summary of Findings 22
Report Analysis – Top 3 Security Issues – Ingrid Francis 23
1. SQL Injection (Highest Severity – Multiple Instances) 23
2. Hard-coded Secrets / Credentials 23
3. Cross-Site Scripting (XSS) Issues 24
Additional Notable Issues and Overall Assessment 25
Additional Finding: Path Traversal and Directory Listing — Róisín Aylward 25
SAST-to-Challenge Correlation — Róisín Aylward 26
Conclusion — Róisín Aylward 26
Dynamic Application Security Testing (DAST) Testing 27
Introduction – Ingrid Francis 27
Methodology – Ingrid Francis 27
Environment Setup 27
Scoping 27
Crawling (Spider Scan) 27
Targeted Active Scan 28

Manual Validation 28
Summary of Findings 29
Detailed Findings and Validation 30
1. SQL Injection (High Severity) – Ingrid Francis 30
2. Missing Content Security Policy (Medium Severity) – Ingrid Francis 31
3. Missing Anti-Clickjacking Header (Medium Severity) – Ingrid Francis 32
4. Application Error Disclosure (Medium Severity) – Róisín Aylward 33
5. Cross Domain Misconfiguration (Medium Severity) – Róisín Aylward 34
6. Session ID in URL Rewrite (Medium Severity) – Róisín Aylward 36
Conclusion – Ingrid Francis 37
References 37
Appendices 39
Appendix A: SAST Scan Report – Róisín Aylward 39
Appendix B: DAST Scan Report (2026-04-27-ZAP-Report.html) – Ingrid Francis 48
Appendix C: Additional Vulnerability Analyses – Ingrid Francis 51
Vulnerability Analysis 5: Login Admin (Injection) 51
Vulnerability Analysis 6: Email Leak (Sensitive Data Exposure) 54
Vulnerability Analysis 7: CAPTCHA Bypass (Broken Anti Automation) – Optional / Short Version 56
Appendix D: Additional Challenges – Ingrid Francis 58

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00168689
Student Name:	Róisín Aylward

Module Information

Module Title:	Web Application Security
Module Code:	COMP H2704

Assessment Information

Assessment Title:	CA3 Project

Declaration

☒ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☒ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☒ I declare that this assessment submission is my own work.

☒ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Róisín Aylward
Date:	26/03/2026

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00176783
Student Name:	INGRID FRANCIS

Module Information

Module Title:	Web Application Security
Module Code:	COMP H2704

Assessment Information

Assessment Title:	CA3 Project

Declaration

✅ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

✅ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

✅ I declare that this assessment submission is my own work.

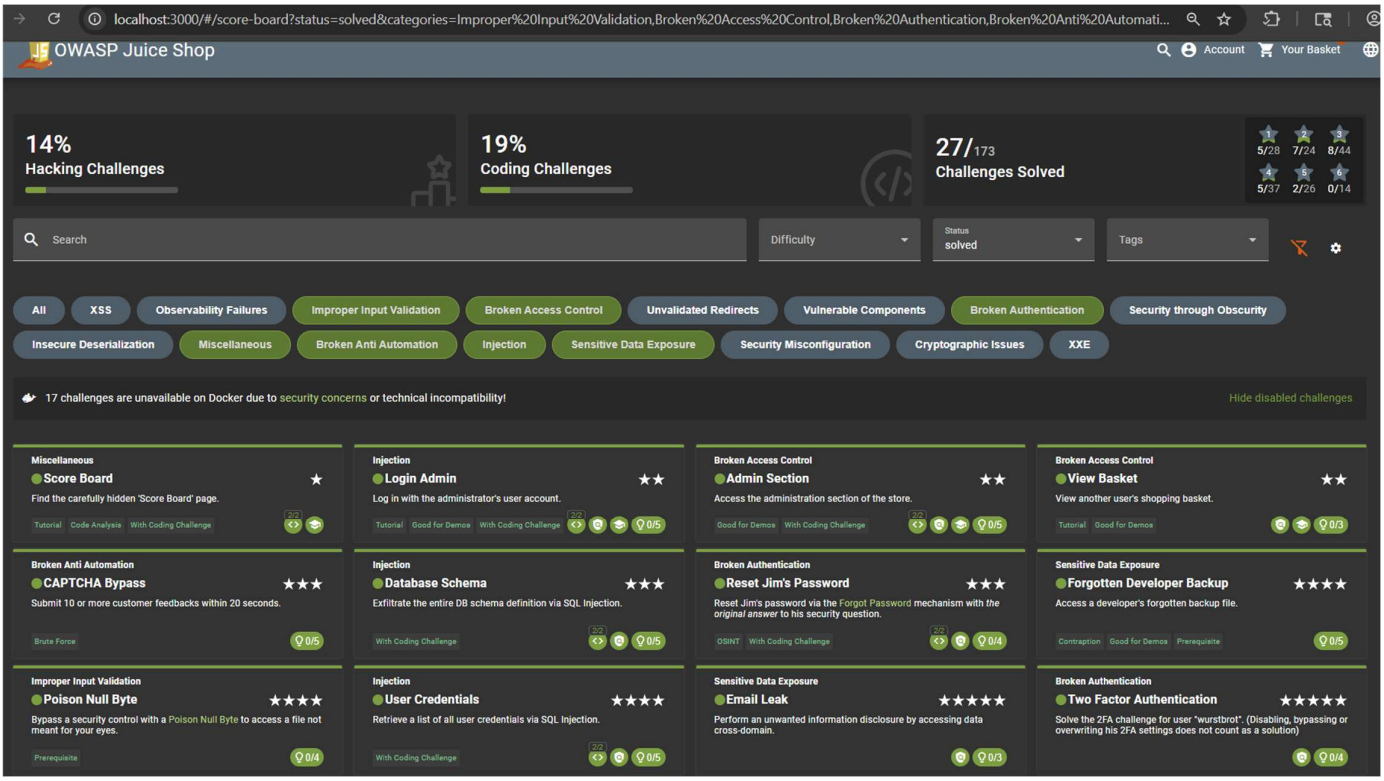
✅ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

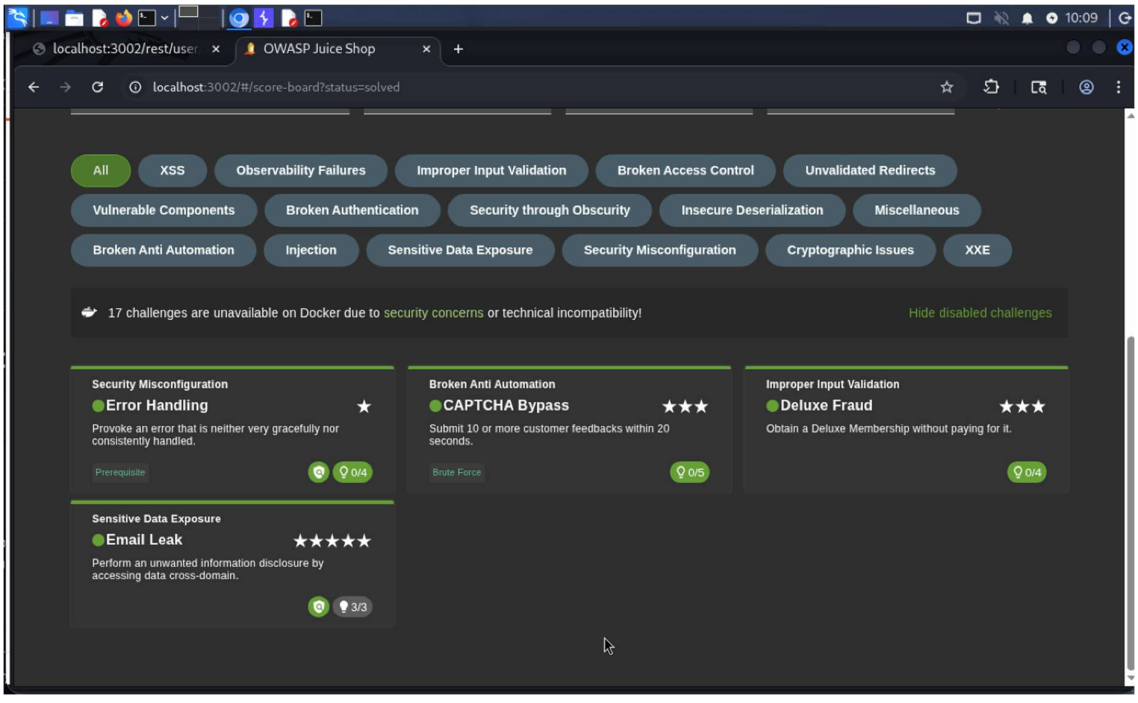
Student signature:	*Ingrid Francis*
Date:	26th April 2026

Total Word Count: 4739 (excluding cover pages, references, figures, and appendices)

Completed Challenges

OWASP Juice Shop Scoreboard screenshots:





Individual Contributions:

Complete a table like the example below to indicate individual contributions.

Challenge ID	Vulnerability	Student Responsible	Evidence	Mitigation Proposed
Admin Section	Broken Access Control	Ingrid Francis	Browser Dev Tools → Sources → main.js (route search) + direct navigation to /#/administration	Server-side RBAC + avoid exposing routes in frontend JS
View Basket	Broken Access Control (IDOR)	Ingrid Francis	Burp Suite intercept – changed bid from 2 to 1	Server-side ownership validation on resource access
Reset Jim's Password	Broken Authentication	Róisín Aylward	OSINT using exported data of reviews and Wikipedia to answer security question	Replace security questions with secure token-based password reset sent by email
Two Factor Authentication	Broken Authentication	Róisín Aylward	totpSecret extracted with SQL injection on Users table and used in TOTP generator to produce 6-digit code	Encrypt TOTP secrets at rest in the database + fix the underlying SQL vulnerability that exposed it
Login Admin	Injection (SQL Injection)	Ingrid Francis	SQL injection payload in login email field (e.g. ' OR 1=1 --)	Parameterized queries + prepared statements
Database Schema	Injection (SQL Injection)	Róisín Aylward	UNION SELECT against sqlite_master /rest/products/search endpoint	Parameterised queries using named placeholders instead of string concatenation
User Credentials	Injection (SQL Injection)	Róisín Aylward	UNION SELECT dumping email and password hash from Users table	Parameterised queries + principle of least privilege on DB user (no SELECT)
CAPTCHA Bypass	Broken Anti Automation	Ingrid Francis	Submitted 10+ customer feedback in under 20 seconds	Rate limiting + proper CAPTCHA enforcement
Email Leak	Sensitive Data Exposure	Ingrid Francis	Accessed /rest/user endpoint cross-domain without authentication	Proper authentication + access control on API endpoints
Forgotten Developer Backup	Sensitive Data Exposure	Róisín Aylward	Null byte bypass (%2500.md) on /ftp/package.json.bak	Remove public /ftp directory exposure

Vulnerability Analyses

Vulnerability Analysis 1: Admin Section (Broken Access Control) – Ingrid Francis

a. **Vulnerability description and root cause** This is a **Broken Access Control** vulnerability (OWASP Top 10 2021: A01:2021, CWE-284: Improper Access Control) [1], [2]. The administration section of the Juice Shop is not shown in the normal user menu. However, the route to this section is still present in the client-side JavaScript code (main.js). The backend does not check properly whether the user has admin rights when someone visits the page directly. This means the application relies only on hiding the link instead of enforcing real security.

b. **Rationale or investigative process** I logged in as a normal user and looked around the application. There was no admin link anywhere in the menu. I then opened the browser Developer Tools by pressing F12, went to the Sources tab, and opened the main JavaScript file. After formatting the code, I searched for the word "administration". This showed me the hidden route. I copied the path and typed it directly into the browser address bar (for example, http://localhost:3002/#/administration). The admin page loaded without any error message, even though I was not an admin. This proved the access control was weak.

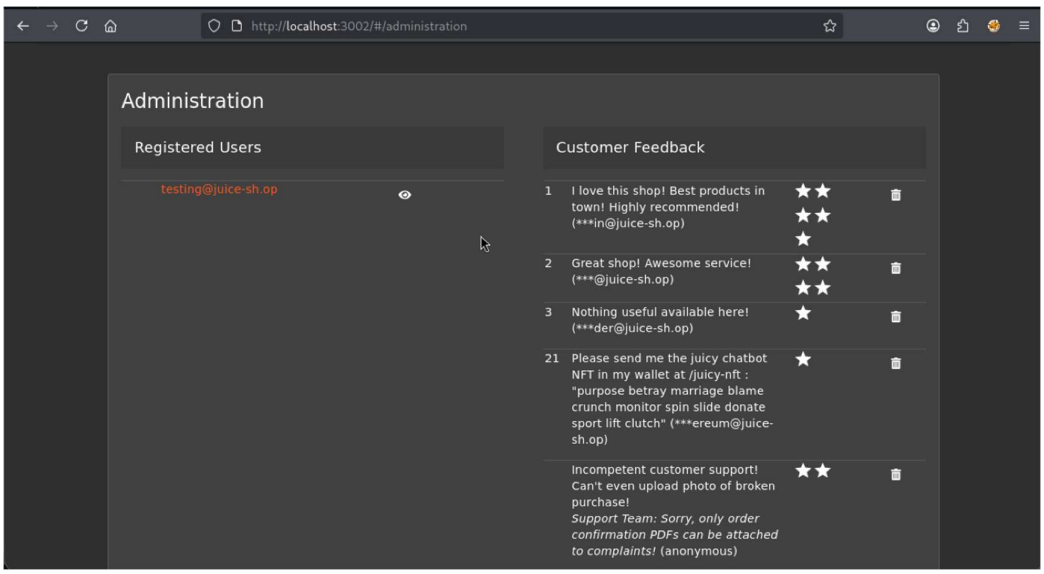


Figure 1 above: /#/administration page accessed as a regular user without admin privileges.

c. **How the exploit works:** The exploit is simple. I inspected the frontend JavaScript code to find the admin route and then navigated straight to it using the browser. The server did not verify my user role and allowed me to see the administration section. Evidence includes screenshots from the Developer Tools showing the route in main.js and the admin page loading successfully as a regular user.

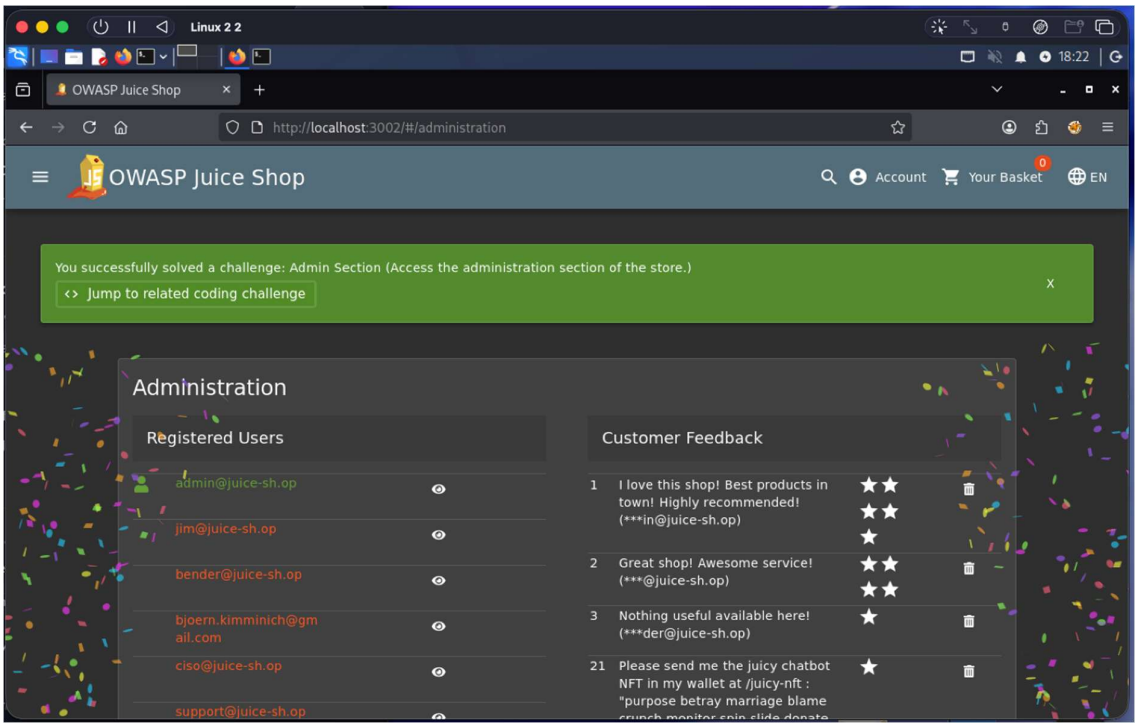


Figure 2 above: administration page successfully accessed as a non-admin user, solving the challenge.

d. **Potential impact** If this happened in a real online shop, any logged-in user could access admin tools. They could change product prices, view all customer information, delete feedback, or even take full control of the system. This could cause serious data breaches, financial losses, and damage to the company's reputation.

e. **Recommended mitigation** The best fix is to add strong **server-side authorization checks** on every admin endpoint. The application should always verify the user's role before showing sensitive pages (OWASP Proactive Controls C7: Enforce Access

Controls) [3]. Sensitive routes should not be left exposed in client-side JavaScript. The server should return a clear 403 Forbidden error for unauthorized users. These steps follow the OWASP Application Security Verification Standard (ASVS) for proper access control [4].

Vulnerability Analysis 2: View Basket (Broken Access Control – IDOR) – Ingrid Francis

a. **Vulnerability description and root cause:**

This is an **Insecure Direct Object Reference (IDOR)** vulnerability, which falls under **Broken Access Control** (OWASP Top 10 2021: A01:2021, CWE-639) [1], [2]. When viewing a shopping basket, the application uses a basket ID (bid) from the request without checking if the logged-in user actually owns that basket.

b. **Rationale or investigative process:**

While I was looking at my own basket, I noticed a basket ID number in the URL or request. Using Burp Suite [5], I intercepted the request and saw my own basket ID (for example, 2). I changed this number to another user's basket ID (for example, 1) and sent the request again. The server returned the other user's basket contents without asking for permission. This showed that the application trusted the ID I provided.

c. **How the exploit works:**

I used Burp Suite to catch the basket request and simply changed the bid value from my own to someone else's. The server did not validate ownership and showed the other user's basket. Evidence includes before-and-after screenshots from Burp Suite showing the modified request and the response containing another user's items.

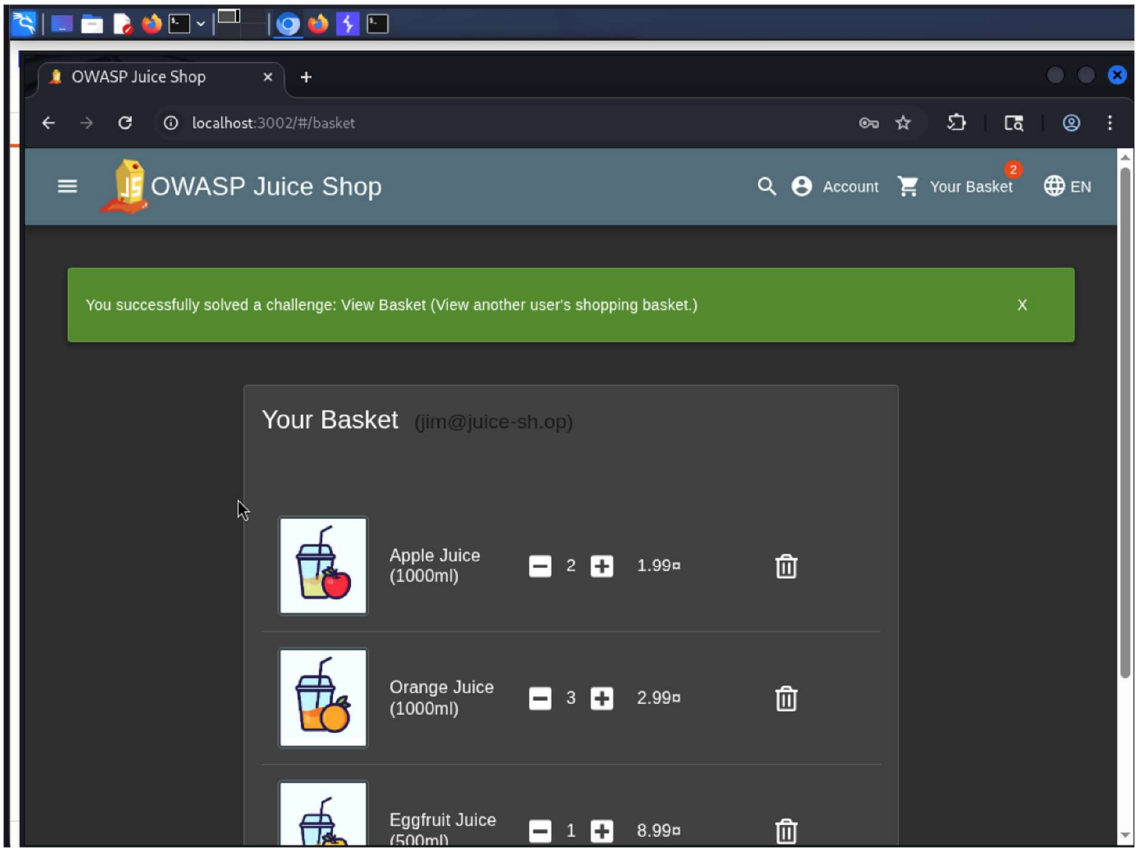


Figure 3 above: jim@juice-sh.op's basket contents accessed after modifying the basket ID in Burp Suite, solving the challenge.

d. **Potential impact:**

In a real e-commerce website, attackers could view other customers' private shopping baskets. This might expose addresses, order history, or even payment details. It would break customer privacy and could lead to identity theft or further attacks.

e. **Recommended mitigation:**

The application must check on the server side that the user owns the requested basket before showing its contents. Using random and hard-to-guess IDs (such as UUIDs) instead of simple numbers helps as an extra layer of protection. Proper authorization checks should be applied to every user-specific resource (OWASP ASVS 4.2 and Proactive Controls C7) [3], [4].

Vulnerability Analysis 3: SQL Injection – Róisín Aylward

a. Vulnerability description and root cause

An SQL Injection is a web application vulnerability classified under OWASP Top 10 2025 as A05: Injection and mapped to CWE-89: Improper Neutralisation of Special Elements used in an SQL Command [2], [6]. It occurs as an attack when a malicious actor manipulates user input (such as a search function in a website or HTTP request) to be interpreted as an SQL query in order to gain unlawful access to a database.

The root cause of the vulnerability in OWASP Juice Shop was identified in the application's source code, specifically in two routes: routes/search.ts and routes/login.ts. Both routes had raw SQL queries constructed by embedding user input directly into the query string using JavaScript template literals rather than safe parameterised methods. For example, the variable "criteria" which contains the user supplied URL parameter in routes/search.ts is inserted into the query without any sanitisation.

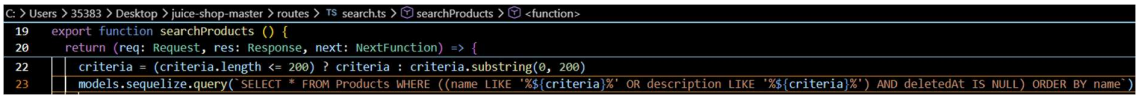


Figure 4 above: Raw SQL query in routes/search.ts embedding user input using template literals without sanitisation.

b. Rationale or investigative process:

The investigation began by reviewing the SAST scan results (see SAST Testing section), which had already flagged routes/search.ts:23 and routes/login.ts:34 at ERROR severity for unsanitised raw query construction. This pointed to the /rest/products/search?q= endpoint as the primary attack surface. Rather than using the search bar on the page, which performs its own frontend handling of the input, the endpoint was targeted directly using the browser address bar to bypass any client-side interference. The DAST scan (see DAST Testing section) independently corroborated this by returning a SQLITE_ERROR response from the same endpoint, confirming that the injected input was being processed by the database engine. Together, the SAST and DAST findings established a clear basis for manual exploitation.

c. How the exploit works:

Two challenges were completed to demonstrate the impact of this vulnerability. The first step was to determine how many columns the original query returns, as UNION-based SQL injection requires the injected query to match the column count exactly. This was done by incrementally adding columns to a UNION SELECT payload in the address bar until the server returned results without an error, confirming nine columns.

The Database Schema challenge was solved by querying the sqlite_master system table:

http://localhost:3000/rest/products/search?q=juice'))UNION SELECT sql,'2','3','4','5','6','7','8','9' FROM sqlite_master--



Figure 5 above: UNION SELECT on sqlite_master returns full CREATE TABLE definitions including the Users table schema.

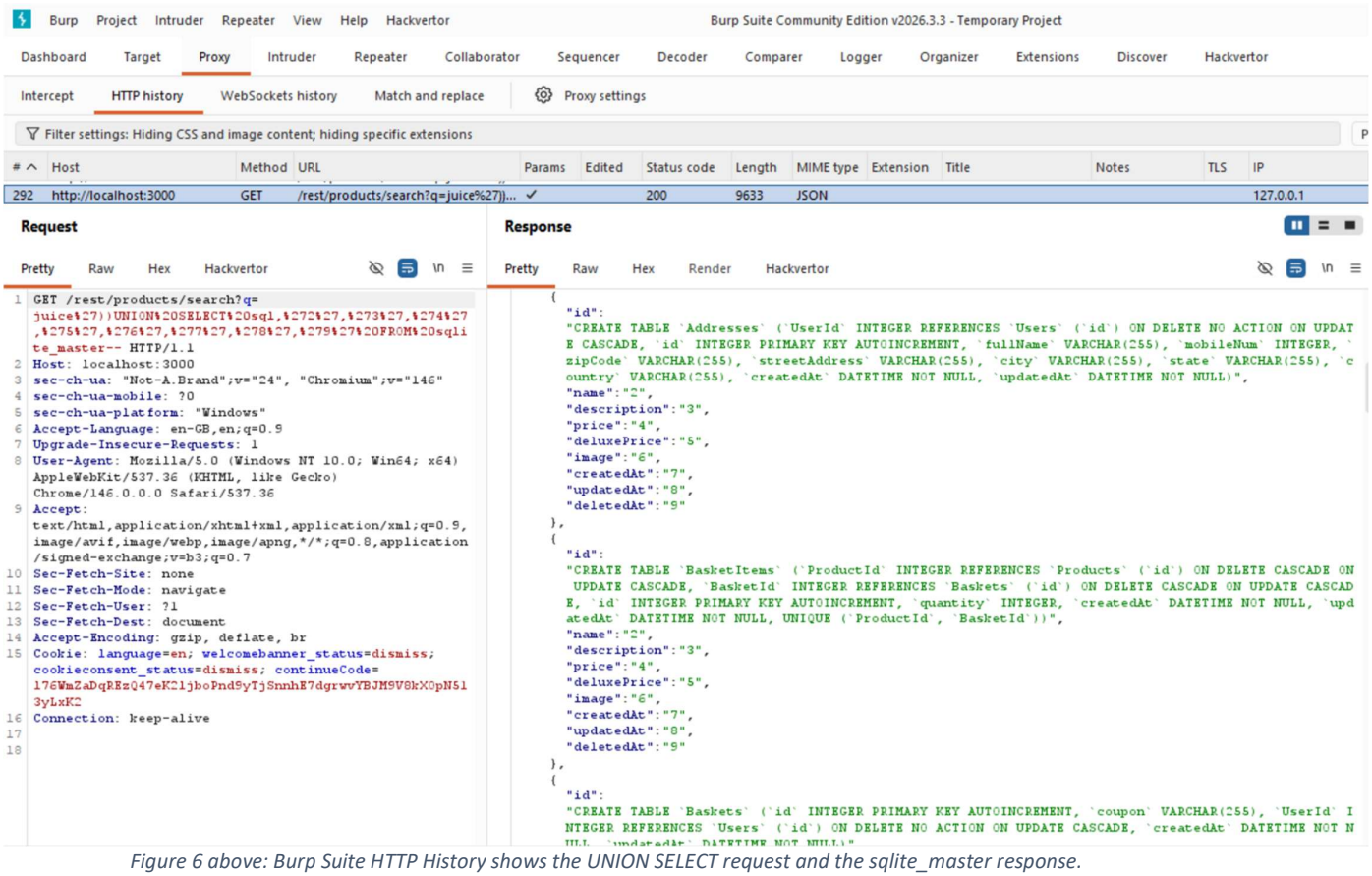


Figure 6 above: Burp Suite HTTP History shows the UNION SELECT request and the sqlite_master response.

The response returned the full CREATE TABLE definitions for every table, including the Users table and its column names: username, email, password, role, lastLoginIp, profileImage,

totpSecret, isActive, createdAt, updatedAt, deletedAt. Knowing the columns helped advance into the User Credentials challenge that required targeting the Users table directly:

http://localhost:3000/rest/products/search?q=juice'))UNION SELECT username,email,password,'4','5','6','7','8','9' FROM Users--

The response returned every username, email address, and passwords stored as MD5 hashes which could be reversed to plaintext using crackstation.net.

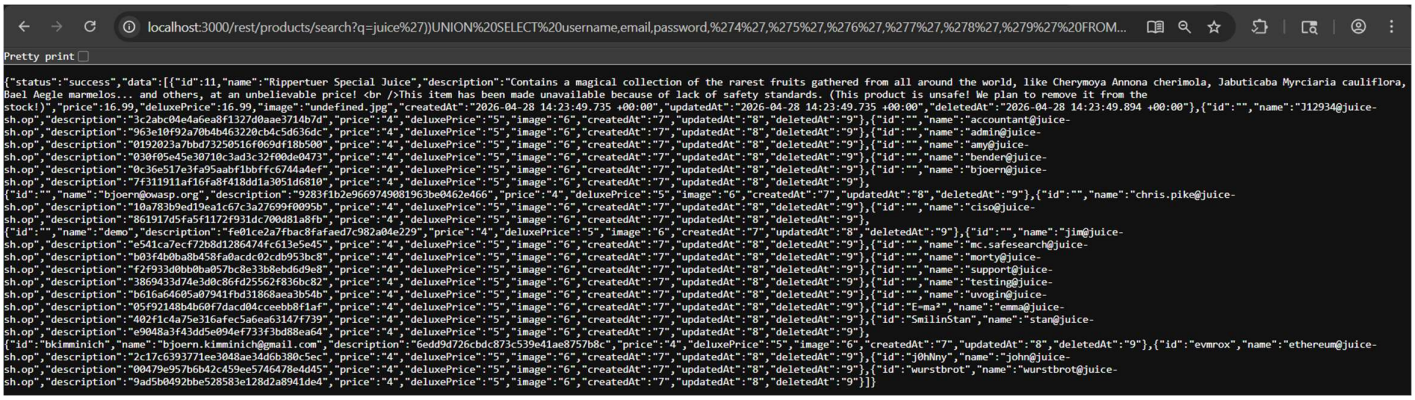


Figure 7 above: UNION SELECT on the Users table returns all usernames, email addresses and MD5 password hashes.

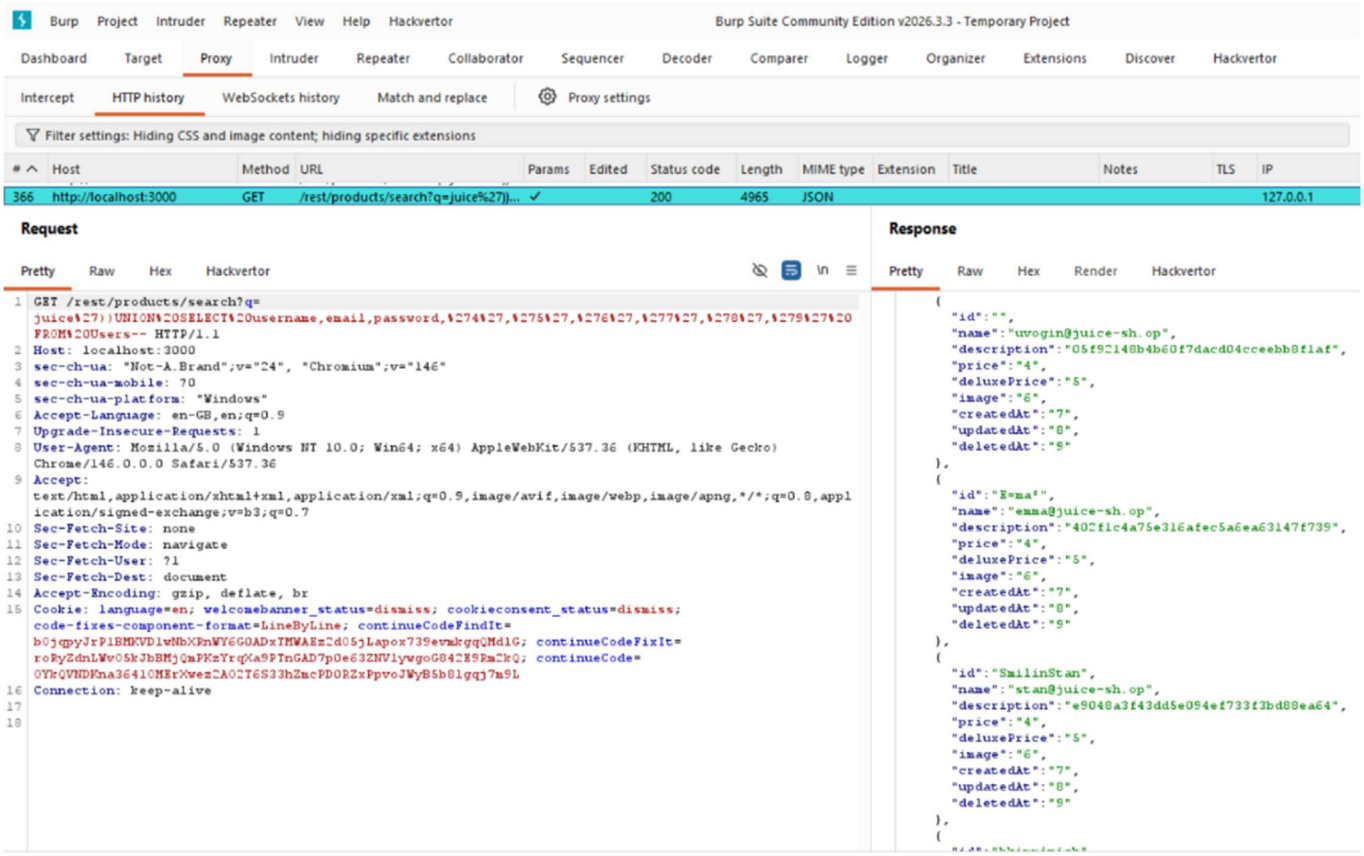


Figure 8 above: Burp Suite HTTP History shows the UNION SELECT request targeting the Users table.

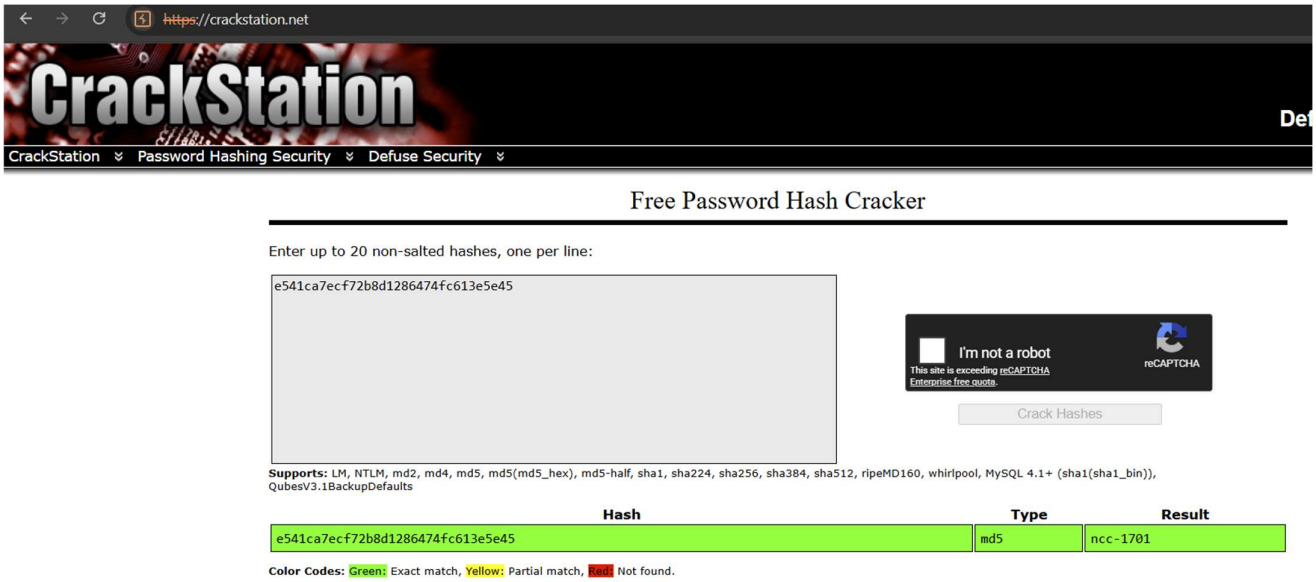


Figure 9 above: CrackStation reverses Jim's MD5 password hash to plaintext password ncc-1701.

d. Potential impact:

If the vulnerability were exploited in a real-world system it could be catastrophic as an attacker could extract the entire contents of the database including user credentials and roles. As MD5 was used for password hashing, the cracked passwords could then be used in credential stuffing attacks against other services where users have reused the same password. The totpSecret column being exposed in plaintext makes two-factor authentication ineffective as it can now be bypassed as demonstrated in the Two Factor Authentication challenge.

e. Recommended mitigation:

According to OWASP Proactive Control C3: Validate all Input & Handle Exceptions [3], all raw string concatenated queries should be replaced with parameterised queries where user input is passed separately from the SQL statement. Additionally, V11.4.1 in OWASP's ASVS (version 5.0.0) [4] in general recommends against using MD5 as a hash function "for any cryptographic purpose" while V11.4.2 states to store passwords in an "approved, computationally intensive, key derivation function" such as bcrypt or Argon2.

Vulnerability Analysis 4: Broken Authentication – Róisín Aylward

a. Vulnerability description and root cause:

Broken Authentication is a web application vulnerability classified under OWASP Top 10 2025 as A07: Authentication Failures and mapped to CWE-287: Improper Authentication [2], [6]. It occurs when weaknesses in an application's authentication allow attackers to impersonate legitimate users by compromising passwords, session tokens, or authentication bypass methods.

Two root causes were identified in Juice Shop. Firstly, the use of knowledge based security questions for password recovery, where the answer to Jim's security question ("Your eldest sibling's middle name?") was discoverable using OSINT. The second is that TOTP secrets used for two factor authentication are stored in plaintext in the Users table, meaning that a database compromise also compromises 2FA.

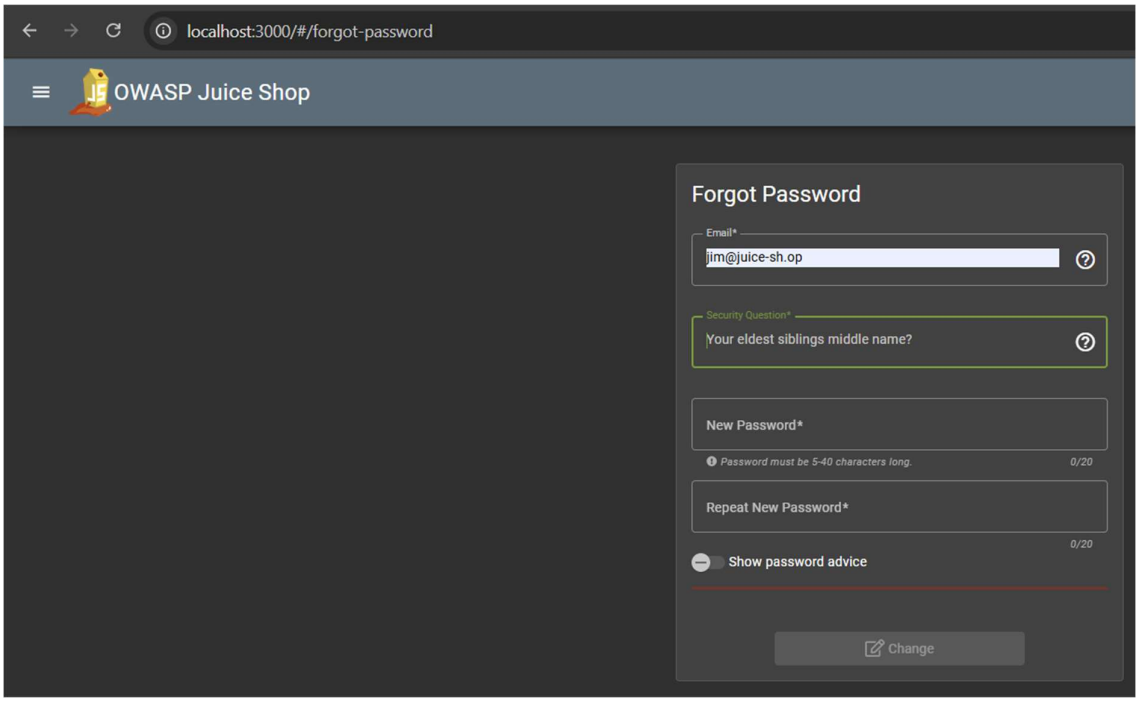


Figure 10 above: Juice Shop's Forgot Password page shows Jim's knowledge based security question.

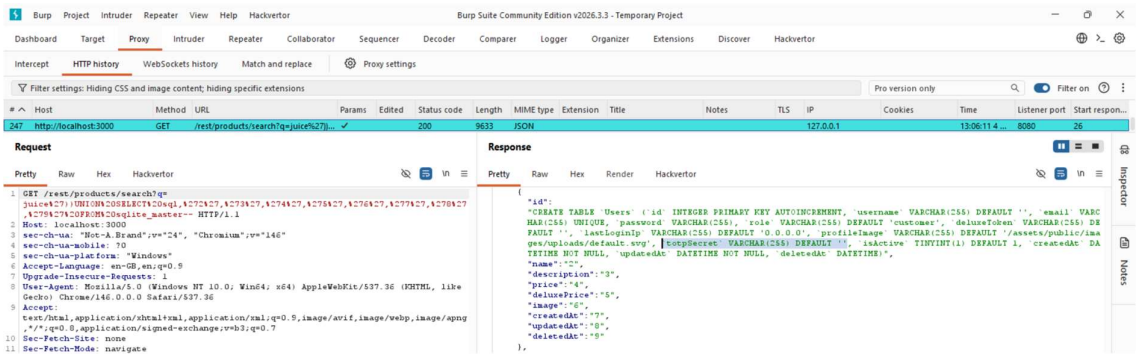


Figure 11 above: Users table column list including totpSecret which is stored as a plain VARCHAR column.

b. Rationale or investigative process:

The investigation began as a direct continuation of the User Credentials challenge. Jim's password hash (e541ca7ecf72b8d1286474fc613e5e45) had already been obtained from the SQL injection dump and reversing it using CrackStation.net [7] and MD5Hashing.net [8] revealed the plaintext password ncc-1701. This allowed login into Jim's account using jim@juice-sh.op before the password reset had even taken place. Once logged in, the Request Data Export function in the account's Privacy & Security section was used to download his account data, which included his product reviews. The reviews mentioned "Starfleet", a reference to Star Trek, identifying the account as James T. Kirk. A search for "Jim Starfleet" found that Kirk's brother is George **Samuel** Kirk [9].

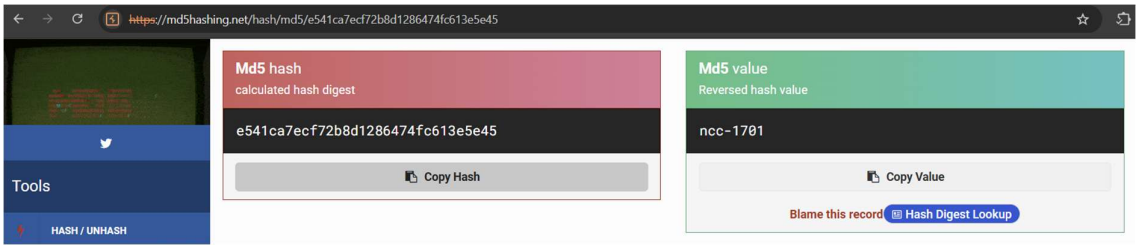


Figure 12 above: MD5Hashing.net also reverses Jim's password hash to ncc-1701, enabling login into his account before the password reset

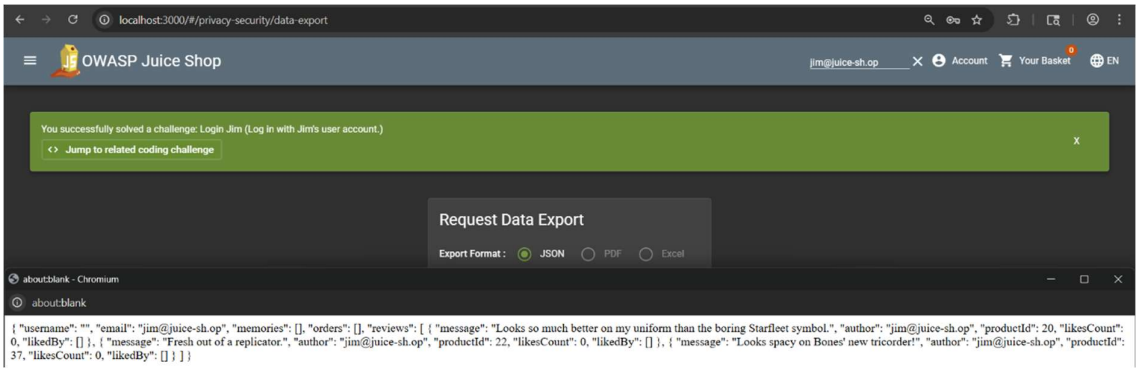


Figure 13 above: Jim's exported account data shows the product reviews containing a Star Trek reference such as "Starfleet."



*Figure 14 above: Wikipedia page for James T. Kirk that shows the brother's name as George **Samuel** Kirk.*

The investigation into the second root cause came from the Database Schema challenge, where the full Users table column list was exposed including the totpSecret column, which made it worth exploring whether that secret could be used to bypass 2FA.

c. How the exploit works:

For the Reset Jim's Password challenge, "Samuel" was entered as the security question on the Forgot Password page for jim@juice-sh.op, which successfully reset his password and completed the challenge. Logging into Jim's account was possible before resetting his password because his original password hash was already cracked through SQL injection.

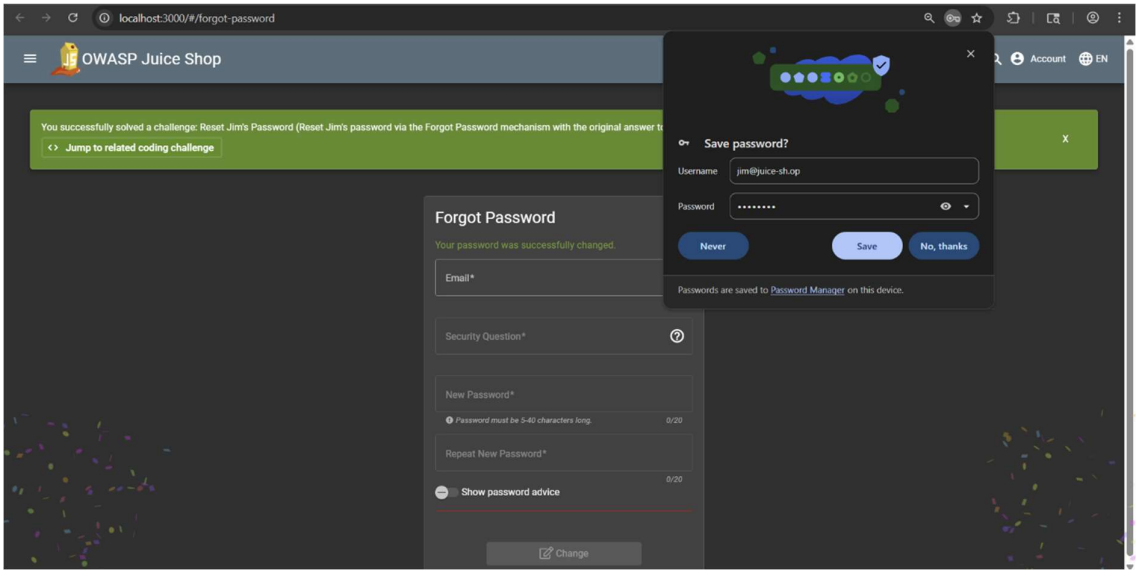


Figure 15 above: Jim's password successfully reset by using the OSINT answer "Samuel".

For the Two Factor Authentication challenge, the totpSecret column was extracted from the Users table using a modified version of the same UNION SELECT payload from the Injection challenges:

http://localhost:3000/rest/products/search?q='))UNION SELECT username,email,totpSecret,'4','5','6','7','8','9' FROM Users--

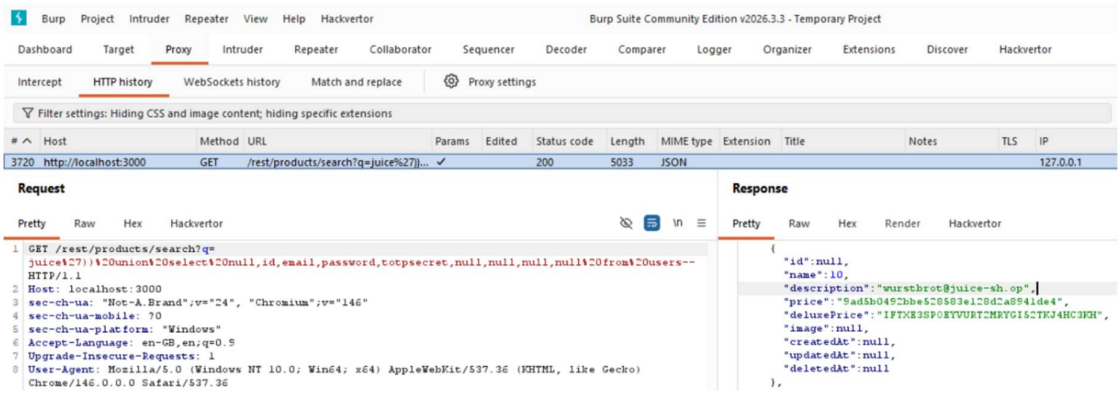


Figure 16 above: UNION SELECT on the Users table shows wurstbrot's totpSecret value.

The base32 encoded TOTP secret for wurstbrot@juice-sh.op was pasted into an online TOTP generator (totp.app) [10], which produced the 6 digit authentication code. This allowed login, passing the 2FA prompt.

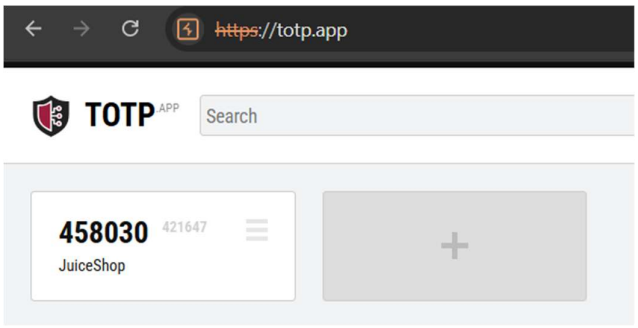


Figure 17 above: totp.app generates a valid 6-digit authentication code from the extracted TOTP secret.

d. Potential impact:

In a real-world system, the password reset vulnerability would allow any account to be taken over by an attacker willing to do basic research on the target. While the method used to reset Jim's account was to first log in and download reviews data, it is possible to manually look through the reviews of every product to find the answer without having to login. The 2FA bypass is particularly damaging as two-factor authentication is specifically used to protect accounts despite passwords being compromised. Storing TOTP secrets in plaintext in the same database as passwords defeats the purpose as an attacker who dumps the database gains both credentials and the ability to bypass 2FA on every account.

e. Recommended mitigation:

V6.4.2-3 in OWASP's ASVS (version 5.0.0) [4] recommends against using knowledge-based authentication such as "secret questions" and to instead implement a "secure process for resetting a forgotten password … that does not bypass any enabled multi-factor authentication mechanisms."

For TOTP secrets, V6.5.2 [4] requires them to be hashed using an approved password storage hashing algorithm with a 32-bit random salt rather than stored in plaintext, so that a database breach alone cannot expose them.

Static Application Security Testing (SAST)

Introduction – Róisín Aylward

Static Application Security Testing (SAST) is used to analyse application source code without executing the application. The aim is to find possible vulnerabilities caused by insecure programming patterns such as unsanitised inputs that could lead to SQL or command injection, hardcoded credentials, usage of insecure cryptography algorithms, and path traversals. It is effective at identifying root causes early and before deployment but suffers from many false positives as it cannot differentiate between vulnerable production code and harmless code that simply matches a dangerous pattern, such as intentional training files or dummy test credentials. Semgrep [11] was used to perform a SAST scan on the OWASP Juice Shop source code [12].

Methodology – Róisín Aylward

Environment Setup

The Juice Shop source code was downloaded from the official GitHub repository [13] and tested on Windows 11 with Docker Desktop installed, and Semgrep run via the official semgrep/semgrep Docker image.

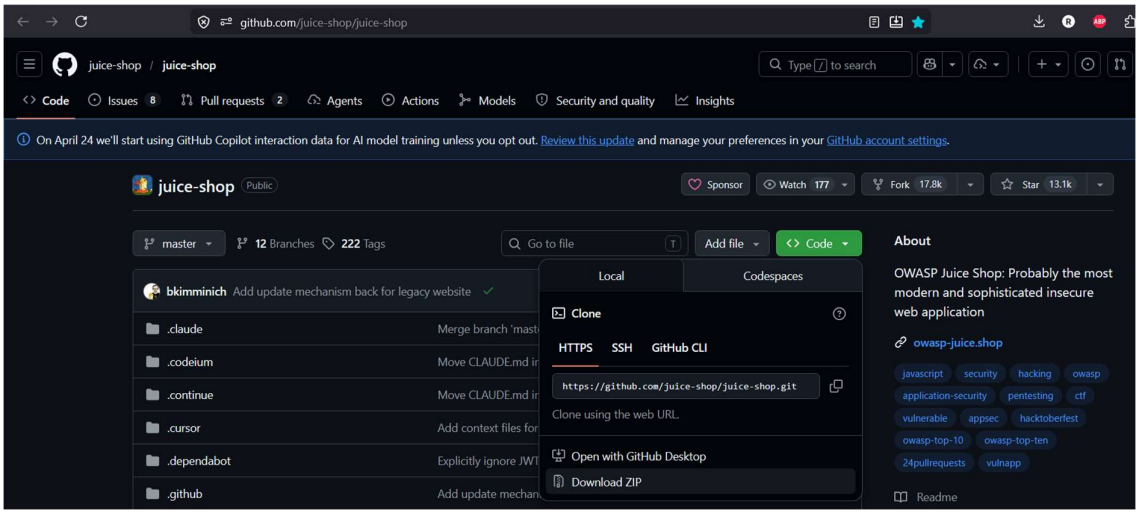


Figure 18 above: GitHub repository of Juice Shop. Copy is downloading.

Running the Scan

Semgrep was run through Docker as native Windows installation is not supported. The following command was executed in PowerShell from the root of the Juice Shop source directory:

```
docker run --rm -v "${PWD}:/src" semgrep/semgrep semgrep --config=auto /src
```

The --config=auto flag automatically selects the most appropriate community security rules for the languages detected. A JSON export was also produced for archival purposes using the --json flag.

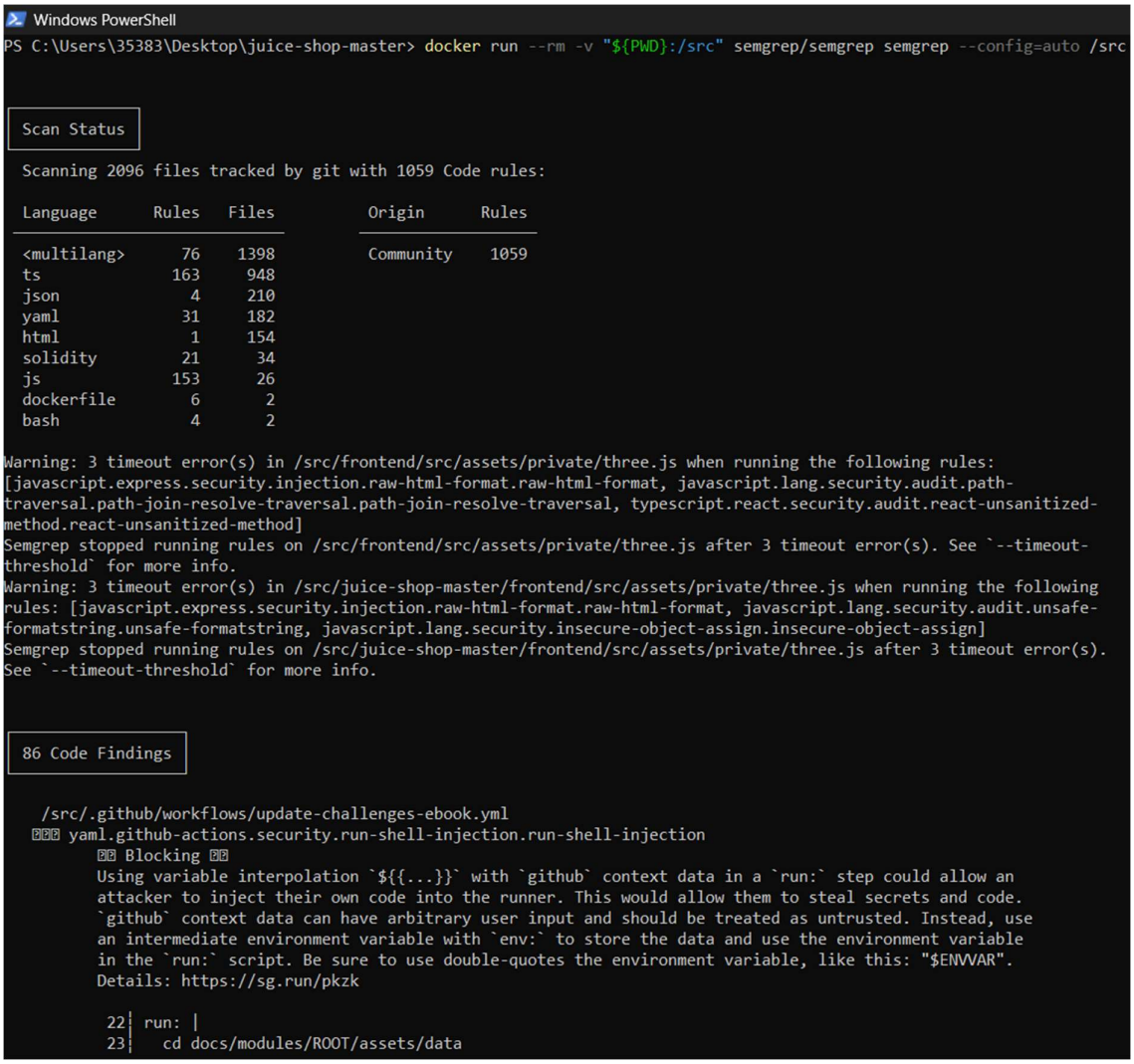


Figure 19 above: Semgrep scan running through Docker on Windows PowerShell, shows initial findings.

Scan Statistics

Metric	Value
Rules applied	302
Files scanned	2096
Total findings	86 (43 unique)
Errors	34 (17 unique)
Warnings	46 (23 unique)
Info	6 (3 unique)

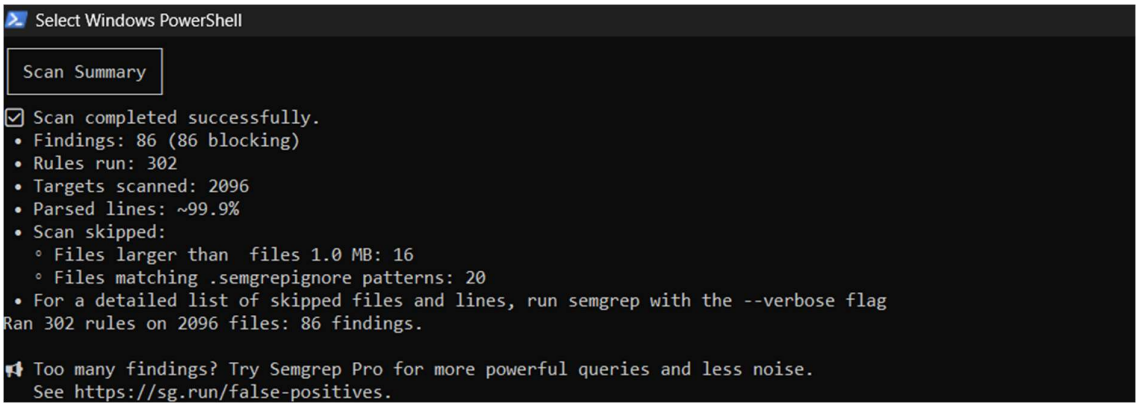


Figure 20 above: Semgrep scan summary.

Summary of Findings

Rank	Vulnerability	Severity	# of Findings	Risk Level	Primary CWE
1	SQL Injection	ERROR	6+	**Critical**	CWE-89
2	Hard-coded Secrets	ERROR/WARNING	5+	**High**	CWE-798
3	XSS (Multiple types)	WARNING	8+	High	CWE-79
4	Path Traversal	WARNING	5	Medium	CWE-22,
5	Directory Listing	WARNING	4	Medium	CWE-548

Report Analysis – Top 3 Security Issues – Ingrid Francis

1. SQL Injection (Highest Severity – Multiple Instances)

Severity: ERROR (High Likelihood + High Impact)

CWE: CWE-89

OWASP: A01:2017 / A03:2021 / A05:2025 – Injection

Affected Files & Locations:

1. data/static/codefixes/dbSchemaChallenge_1.ts
2. data/static/codefixes/dbSchemaChallenge_3.ts
3. data/static/codefixes/unionSqlInjectionChallenge_1.ts
4. data/static/codefixes/unionSqlInjectionChallenge_3.ts
5. routes/login.ts
6. routes/search.ts

Description:

Semgrep detected Sequelize queries that are tainted by unsanitized user input. The queries are built using string concatenation or unsafe raw queries instead of parameterized queries or replacements/bindings.

Why Critical:

These are classic SQL injection vulnerabilities. In several challenges (including login-related ones), successful exploitation could lead to authentication bypass, data exfiltration, or full database compromise.

Recommendation:

Replace unsafe queries with proper parameterization:

- Use replacements / bind parameters in Sequelize
- Prefer the model-based ORM methods (findOne, findAll, etc.) over raw queries when possible

2. Hard-coded Secrets / Credentials

Severity: ERROR / WARNING (Mixed but several High-confidence findings)

Key Findings:

- **Generic Secret** in data/static/users.yml (line 151)
- **Hard-coded JWT tokens** in test files:

    - frontend/src/app/app.guard.spec.ts
    - frontend/src/app/last-login-ip/last-login-ip.component.spec.ts
- **Hard-coded HMAC key** in lib/insecurity.ts (multiple locations)
- **Hard-coded JWT secret** in lib/insecurity.ts:56

CWE: CWE-798 (Use of Hard-coded Credentials), CWE-321

OWASP: A07 – Identification and Authentication Failures

Why Serious:

Hard-coded secrets in source code (especially JWT secrets and HMAC keys) are extremely dangerous. If the code is public or leaked, attackers can forge authentication tokens, impersonate users, or decrypt sensitive data.

Recommendation:

- Move all secrets to environment variables
- Use secret management tools/vaults in production
- Remove or rotate any hard-coded JWT/HMAC keys immediately

3. Cross-Site Scripting (XSS) Issues

Severity: WARNING / INFO (but several high-risk patterns)

Main Findings:

- **Manual sanitization using replaceAll()** in data/static/codefixes/restfulXssChallenge_2.ts (lines 49)
- **Raw HTML construction** with user data in routes/chatbot.ts:205
- **Unsafe eval() usage** in:
    - routes/captcha.ts
    - routes/userProfile.ts (also flagged under code-string-concat)
- **Template unescape** in views/promotionVideo.pug
- **User-controlled data in <script> tags** in routes/videoHandler.ts

CWE: CWE-79

OWASP: A07:2017 / A03:2021 – Cross-Site Scripting

Why Important:

XSS vulnerabilities allow attackers to execute arbitrary JavaScript in users' browsers, leading to session hijacking, defacement, or further attacks (especially dangerous in an e-commerce-like app).

Recommendation:

- Use established libraries: DOMPurify (client-side) or sanitize-html (server-side)
- Avoid eval(), innerHTML, and manual escaping
- Never trust user input when rendering HTML or JavaScript

Additional Notable Issues and Overall Assessment

Additional Notable Issues (Honorable Mentions)

- **Open Redirect** in routes/redirect.ts
- **Prototype Pollution** risk in frontend/src/hacking-instructor/helpers/helpers.ts
- **ReDoS** (Regular Expression Denial of Service) potential in lib/codingChallenges.ts

Overall Assessment:

The application contains several **high-severity, exploitable vulnerabilities** (especially SQLi and hard-coded secrets), which is expected in an *intentionally vulnerable* training application like Juice Shop. However, some of these issues appear in codefix challenge files, suggesting they are part of the learning exercises.

Priority Fix Order:

1. Fix SQL Injection in production paths (routes/login.ts, routes/search.ts)
2. Remove / externalize all hard-coded secrets
3. Replace unsafe XSS patterns with proper sanitization libraries

Additional Finding: Path Traversal and Directory Listing — Róisín Aylward

Severity: WARNING

CWE: CWE-22: Path Traversal and CWE-548: Exposure of Information Through Directory Listing

OWASP Top 10:2025: A01: Broken Access Control and A02 - Security Misconfiguration

Affected files: routes/fileServer.ts:33, routes/keyServer.ts:14, routes/logfileServer.ts:14, routes/quarantineServer.ts:14, server.ts:269,273,277,281

Semgrep flagged four route files for passing user-supplied filenames input directly to Juice Shop's web framework's res.sendFile() without validating the path first. Additionally, server.ts was flagged at four locations for enabling directory listing, indicating that server directories are openly browsable without authentication.

These two findings together help understand better how the Forgotten Developer Backup challenge completed during this project works. The directory listing finding explains why navigating to http://localhost:3000/ftp exposed a browsable file listing. The res.sendFile() finding explains why the null byte bypass (%2500.md) was effective as the extension filter and the file server handled the filename differently, with the server having stopped reading the filename at the null byte and serving package.json.bak regardless of the .md suffix appended.

Recommendations: Disable directory listing by removing the serveIndex middleware in server.ts. User-supplied filenames should never be passed directly to file-serving functions, instead an explicit allowlist of permitted files should be used, with resolving the full path using path.resolve() to prevent traversal.

SAST-to-Challenge Correlation — Róisín Aylward

The SQL injection finding at routes/search.ts:23 pointed to the /rest/products/search?q= endpoint as the primary attack surface, which was then exploited in the Database Schema and User Credentials challenges using UNION SELECT payloads. The path traversal and directory listing findings at routes/fileServer.ts:33 and server.ts:269–281 provided the technical explanation for the Forgotten Developer Backup challenge, where the null byte filter bypass exploited the exact mismatch between the extension filter and the file server that Semgrep had flagged.

Conclusion — Róisín Aylward

The Semgrep scan identified 86 findings across 302 rules. Because Docker mounted both the root source folder and a subfolder (juice-shop-master) at the same time, every file was scanned twice resulting in 43 unique findings covering SQL injection, hardcoded secrets, path traversal, directory listing, XSS, and several lower-priority issues. The most critical findings, SQL injections in routes/search.ts and routes/login.ts and hardcoded cryptographic keys in lib/insecurity.ts were confirmed as exploitable vulnerabilities through manual testing. Some findings such as the SQL injection flags in data/static/codefixes/ files were identified as false positives upon review as these are intentional coding challenge files rather than live application code.

Dynamic Application Security Testing (DAST) Testing

Introduction – Ingrid Francis

Dynamic Application Security Testing (DAST) is used to assess a running application by interacting with it from the outside, similar to how an attacker would. Unlike SAST, which looks at source code, DAST focuses on how the application behaves in real time, including server responses and possible misconfigurations.

For this assessment, OWASP ZAP 2.17.0 [14] was used to carry out a controlled scan on a locally hosted OWASP Juice Shop instance at:

http://localhost:3002

The aim was to identify security vulnerabilities, test them manually, and evaluate their potential impact.

Methodology – Ingrid Francis

Environment Setup

The testing environment included:

- Kali Linux (running on UTM via macOS)
- OWASP ZAP 2.17.0
- OWASP Juice Shop running locally on port 3002
- Firefox for manual testing
- ZAP HTML report generated after scanning

Scoping

To keep the testing safe and controlled, only the Juice Shop application was included in scope. This was done by adding it to the default context in ZAP:

Include in Context → Default Context

This ensured that no external websites were scanned.

Crawling (Spider Scan)

A Spider scan was used to explore the application and find available endpoints.

Some of the main routes discovered were:

- /rest/products/search

- /api/
- /socket.io/
- Static files under /assets/

Targeted Active Scan

Instead of running a full automated scan, a targeted Active Scan was performed on specific endpoints:

- /rest/products/search
- /api/
- /socket.io/

This scan generated 11 alerts across different vulnerability categories.

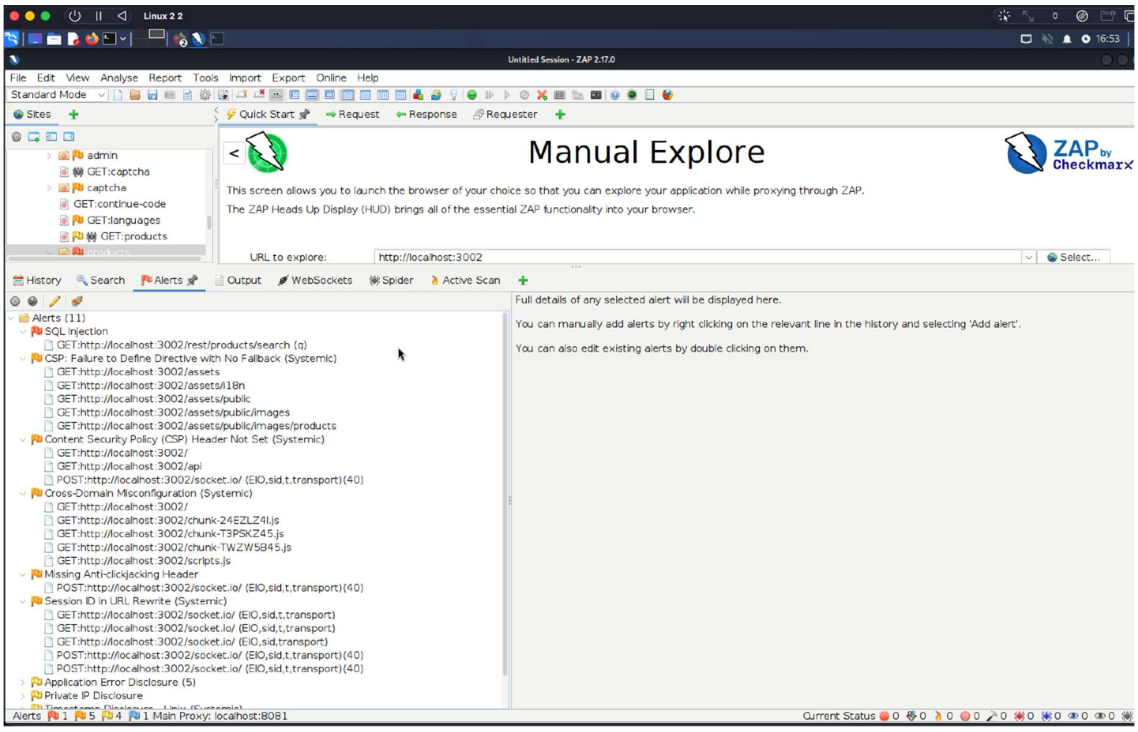


Figure 21 above: ZAP scan report.

Manual Validation

Key vulnerabilities were manually tested using Firefox.

Screenshots were taken as evidence.

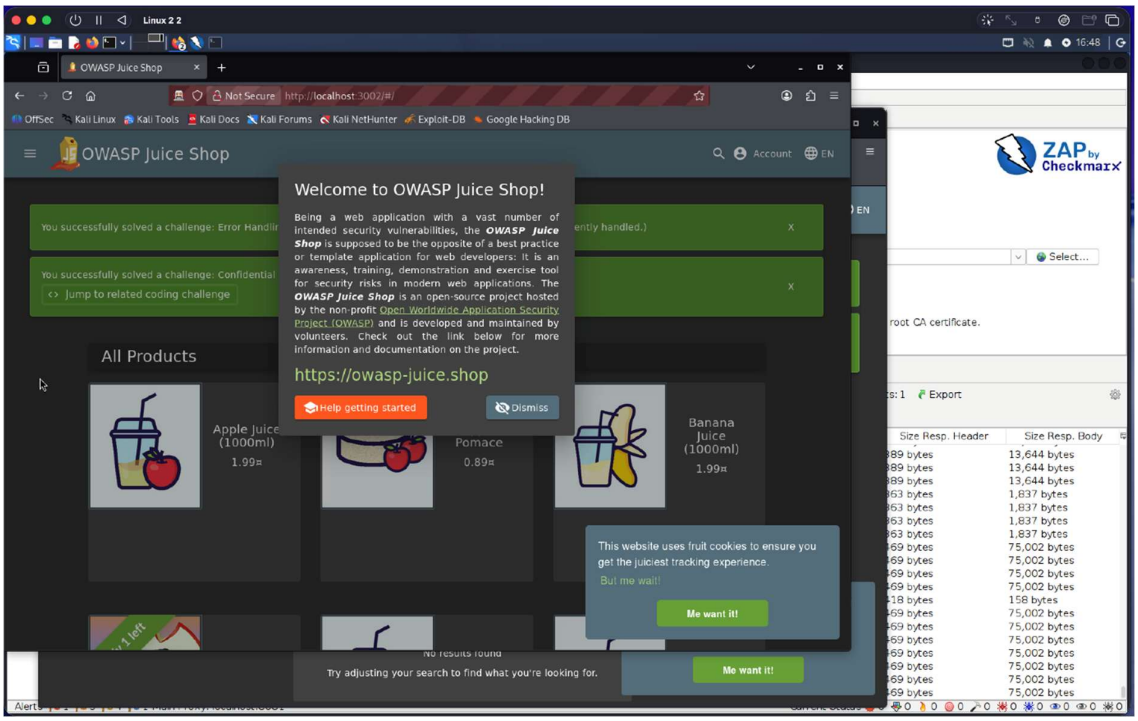


Figure 22 above: Juice Shop on ZAP's launched browser.

Summary of Findings

ID	Vulnerability	Severity	Endpoint	Primary CWE
1	SQL Injection	High	/rest/products/search	CWE-89
2	Missing Content-Security-Policy Header	Medium	/	CWE-693
3	Missing Anti-Clickjacking Header	Medium	/socket.io/	CWE-1021
4	Application Error Disclosure	Medium	/api/	CWE-550
5	Cross-Domain Misconfiguration	Medium	Various	CWE-942
6	Session ID in URL Rewrite	Medium	/socket.io/	CWE-598

Detailed Findings and Validation

1. SQL Injection (High Severity) – Ingrid Francis

Endpoint:
GET /rest/products/search?q=…

Description:
The application does not properly handle user input in the search function. This allows SQL code to be injected into the query.

Manual Test:
The following input was entered in the browser:
http://localhost:3002/rest/products/search?q=' OR 1=1--

Result:
The server returned an error:
SQLITE_ERROR: incomplete input
This shows that the input was processed as part of an SQL query, confirming the vulnerability.

Impact:

- Attackers could access or extract database data
- Search functionality could be bypassed
- Possible login bypass in other areas

Mitigation:

- Use parameterised queries
- Validate all user input on the server side
- Avoid building SQL queries using string concatenation

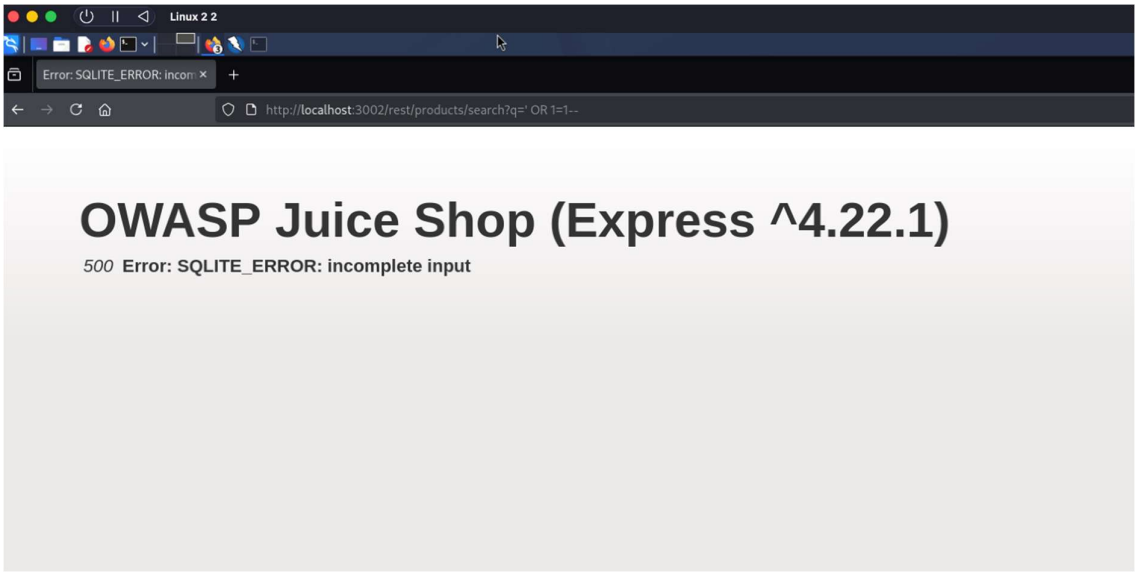


Figure 23 above: Manual SQL Injection test showing database error.

2. Missing Content Security Policy (Medium Severity) – Ingrid Francis

Description:
The application does not include a Content Security Policy (CSP) header [15], which helps protect against cross-site scripting (XSS).

Manual Validation:
Using Firefox Developer Tools, the response headers for the main page (/) were checked.

Result:
No Content-Security-Policy header was found.

Impact:

- Makes XSS attacks easier
- Allows unsafe scripts to run in the browser
- Increases risk of malicious content injection

Mitigation:
Add a CSP header such as:
Content-Security-Policy: default-src 'self';

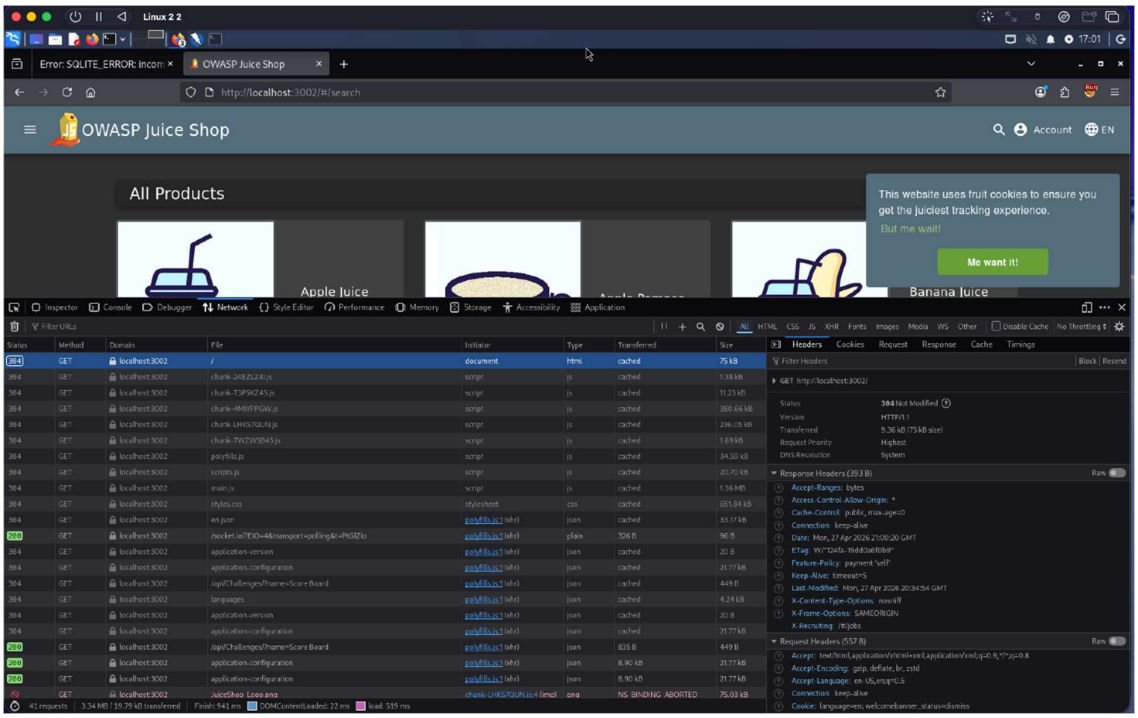


Figure 24 above: Missing CSP header in response.

3. Missing Anti-Clickjacking Header (Medium Severity) – Ingrid Francis

Description:
The application does not include protection against clickjacking attacks.

Manual Validation:
Response headers for /socket.io/ were checked using Firefox Developer Tools.

Result:
The X-Frame-Options header was not present [16].

Impact:

- The site can be embedded in malicious iframes
- Users could be tricked into clicking hidden elements
- Increases risk of clickjacking attacks

Mitigation:
Add one of the following headers:
X-Frame-Options: DENY
or
Content-Security-Policy: frame-ancestors 'none';

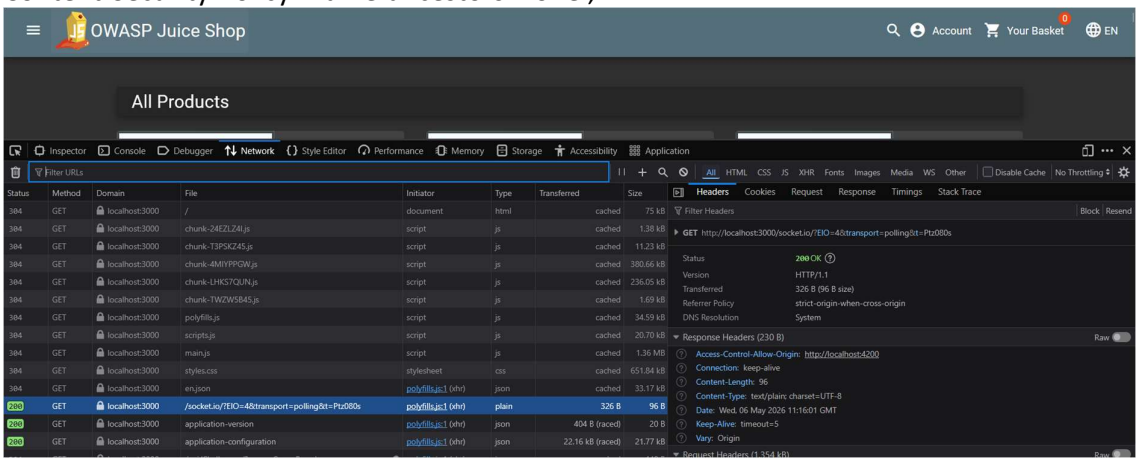


Figure 25 above: Missing anti-clickjacking protection.

4. Application Error Disclosure (Medium Severity) – Róisín Aylward

Description: When an unexpected path is requested at the /api/ endpoint, the server returns a full stack trace, an ordered list of function calls that were made before it the application failed, instead of a generic error page. As a result, internal details about the application that should never be visible to an end user or attacker are exposed. Described in CWE-550: Server-generated Error Message Containing Sensitive Information.

Manual Validation: Entering http://localhost:3000/api/ in the browser caused a 500 Internal Server Error response containing the full Node.js stack trace.

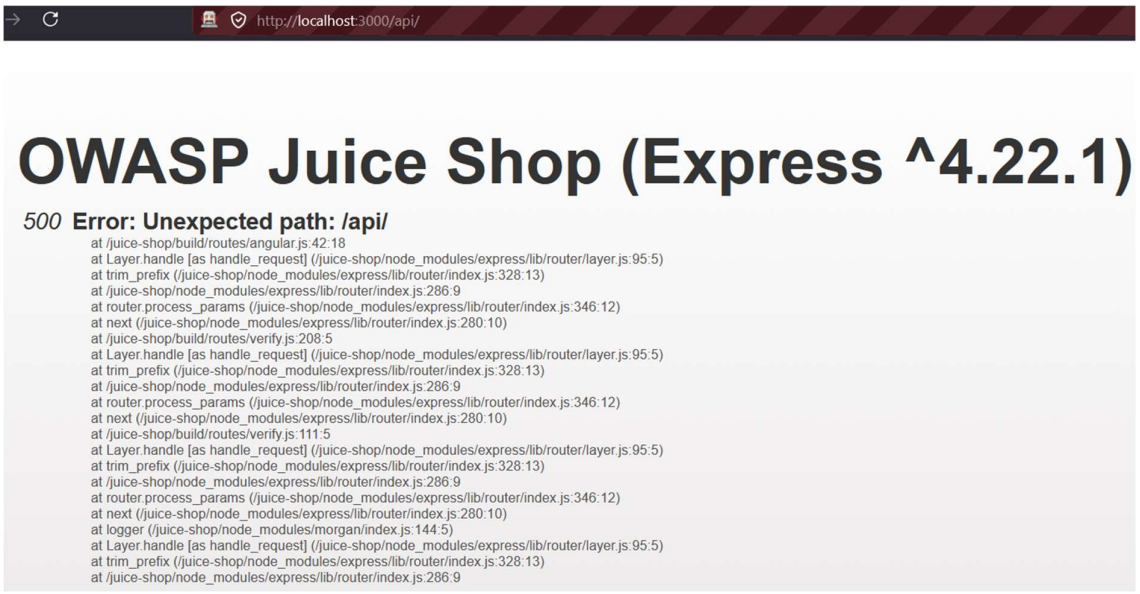


Figure 26 above: Full Node.js stack trace returned by /api/ endpoint exposes Express version and internal file paths.

Result: The response revealed the application's Express framework version as 4.22.1. It also exposed the internal directory structure by showing paths such as:

- /juice-shop/build/routes/angular.js
- /juice-shop/build/routes/verify.js
- /juice-shop/node_modules/express/lib/router/

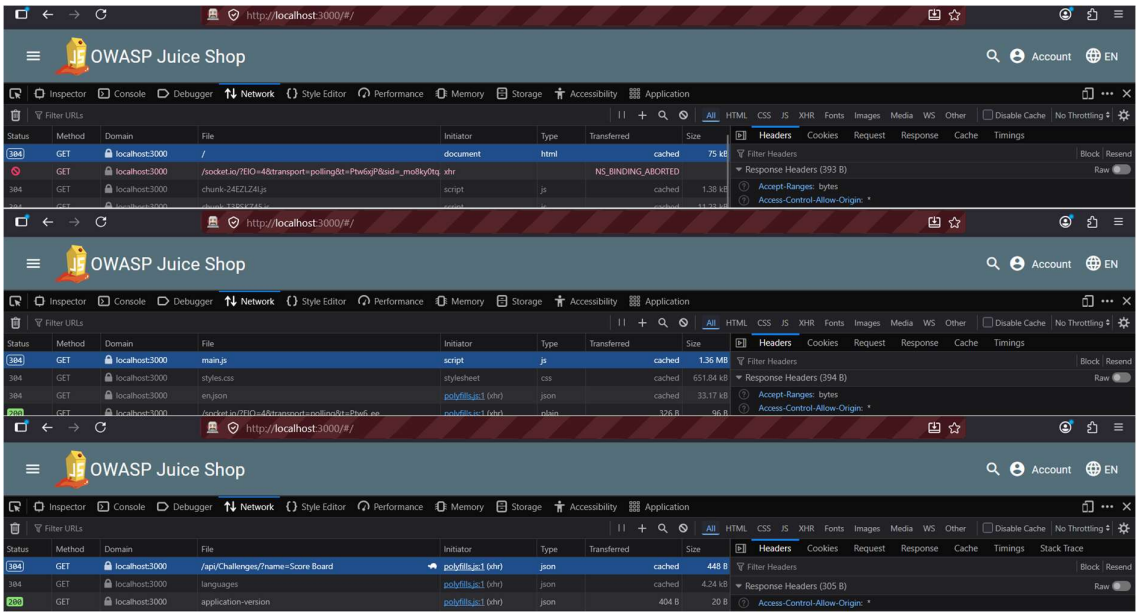
Impact: The shown Express version allows an attacker to search for known CVEs targeting that specific version. The exposed internal file paths reveals where the application's routing logic lives, which decides which code to run in response to URL requests. In a real world system this information would make it easier to identify further attack surfaces, reducing the reconnaissance work needed to launch a targeted attack.

Mitigation: OWASP's ASVS V16.5.1 [4] states that applications should return generic error messages and never expose internal error details that includes "sensitive data such as stack traces, queries, secret keys, and tokens" to the client. Express supports this through a dedicated error-handling middleware that catches unhandled errors and responds with a safe message.

5. Cross Domain Misconfiguration (Medium Severity) – Róisín Aylward

Description: Web servers use Cross-Origin Resource Sharing (CORS) to allow specified domains to access restricted resources through HTTP response headers. The ZAP scan detected a CORS misconfiguration in Juice Shop's server as the application sets the Access-Control-Allow-Origin: * header across all of its responses, allowing any website on the internet to make requests to the application and read the responses rather than only trusted domains. This weakness is classified as CWE-942: Permissive Cross-domain Policy with Untrusted Domains.

Manual Validation: The Access-Control-Allow-Origin: * header was confirmed by checking the Response Headers to any request to http://localhost:3000 that is listed in Firefox's DevTools' Network tab.



*Figures 27-29 above: Access-Control-Allow-Origin: * header in multiple endpoints.*

Result: The wildcard value (*) appears in all responses regardless of the endpoint, meaning there are no restriction on which external domains can interact with the application.

Impact: In a real-world system with authenticated endpoints returning sensitive user data, this misconfiguration could allow a malicious website to make requests to Juice Shop on behalf of a logged-in user and read the responses. This is demonstrated using the /api/Hints/ endpoint which returns 93KB of internal application data. A targeted attack could use this to quietly extract sensitive data on individuals or the application system from authenticated API endpoints if a victim visits a malicious page while logged into the shop.

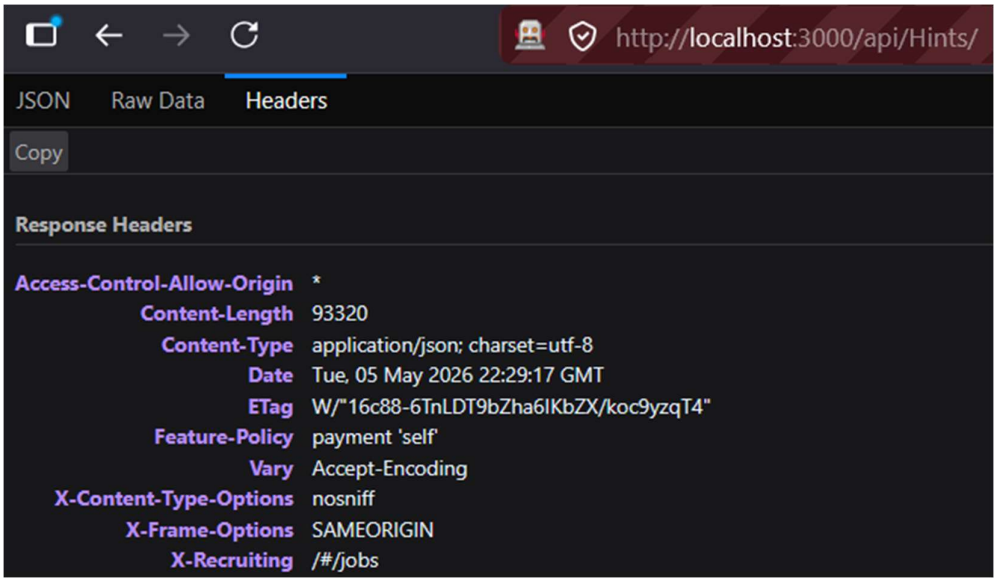


Figure 30 above: /api/Hints/ response header shows endpoint returns 93KB of data.

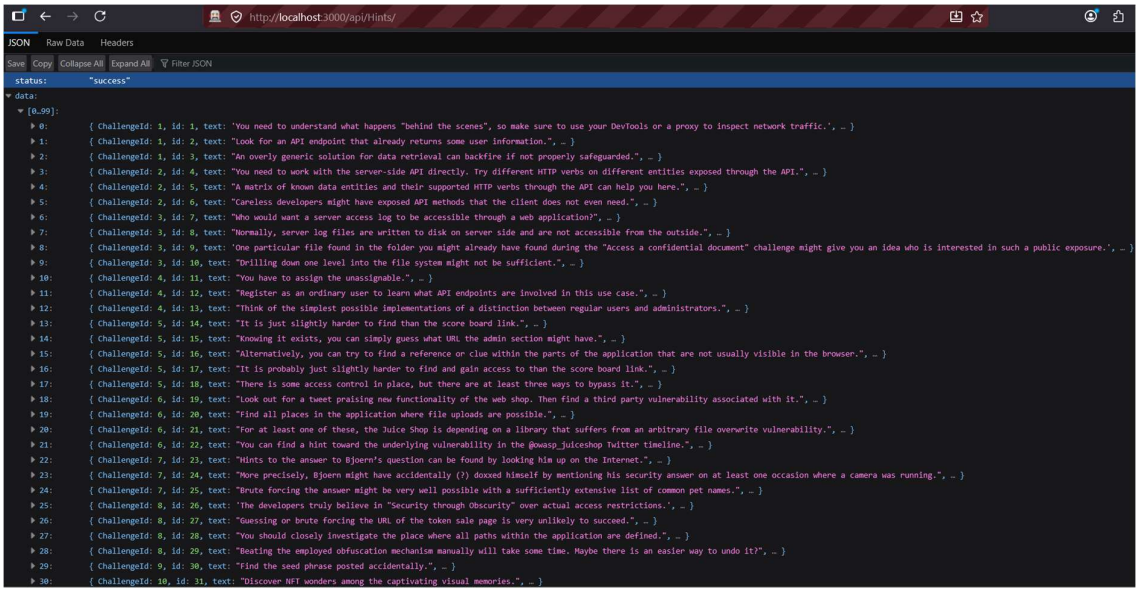


Figure 31 above: /api/Hints/ endpoint showing internal challenge hint data without authentication.

Mitigation: ASVS V3.4.2 [4] recommends the CORS Access-Control-Allow-Origin header field to be a fixed value by the application rather than use the wildcard, or to at least verify that the response does not include any sensitive information if the website's resources must be accessible to all.

6. Session ID in URL Rewrite (Medium Severity) – Róisín Aylward

Description: Session IDs are unique tokens that identify a user's active connection to a server. They should only ever be transmitted via cookies and never in URLs. The ZAP scan detected that Juice Shop's WebSocket connection at /socket.io/ includes the session ID as a sid= parameter directly in the URL. This is classified under CWE-598: Use of GET Request Method with Sensitive Query Strings.

Manual Validation: After hard refreshing the page with DevTools open, the Network tab filtered by "socket.io" showed multiple requests with the sid= parameter visible in the URL.

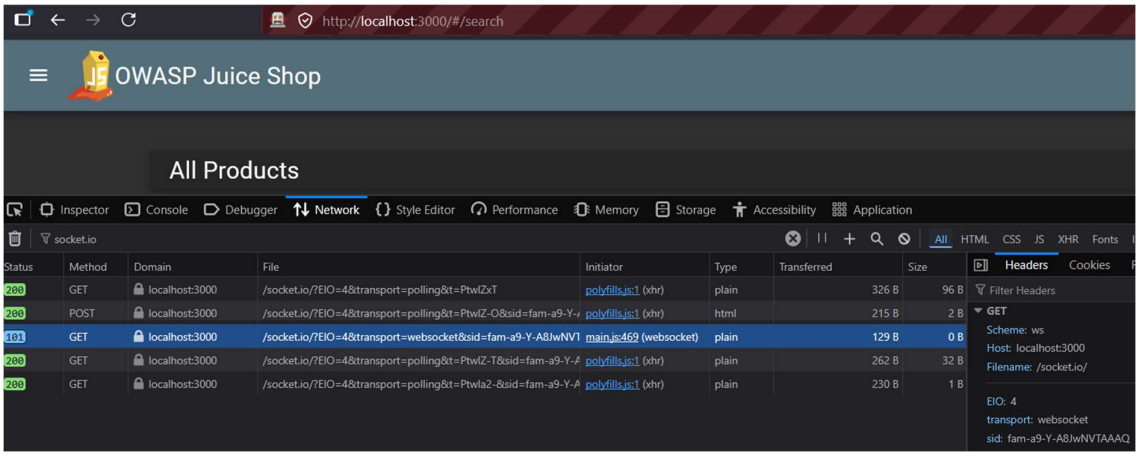


Figure 32 above: Session ID (sid=) visible in socket.io requests.

Result: The session ID appears in plaintext in the URL across multiple socket.io requests. This indicates that it is being transmitted as a query string parameter and not through a secure cookie.

Impact: As Session IDs exposed in URLs are stored in the browser history, anyone who has access to the browser can retrieve them. Session IDs also appear in server access logs and can be leaked to external servers through the HTTP header if the page loads any external resources. An attacker who obtains a valid session ID could use it to hijack the user's active session without needing their password.

Mitigation: To prevent session tokens from leaking through browser history or HTTP headers, and being exposed in URLs, ASVS V3.3.4 [4] states that the value of cookies are "not meant to be accessible to client-side scripts (such as a session token)" and "must only be transferred to the client via the 'Set-Cookie'header field" with the HttpOnly attribute set. V3.3.1 [4] additionally requires the Secure attribute set to ensuring cookies are only sent over encrypted connections.

Conclusion – Ingrid Francis

The DAST testing successfully identified several security issues in the Juice Shop application. By combining automated scanning with manual testing, it was possible to confirm real vulnerabilities such as SQL Injection and missing security headers.
These results show how important runtime testing is, as it can reveal issues that may not be visible in source code alone.

References

[1] OWASP Foundation, "OWASP Top 10:2021," *OWASP*, [Online]. Available: https://owasp.org/Top10/2021/. [Accessed 26 April 2026].

[2] MITRE Corporation, "Common Weakness Enumeration (CWE)," *MITRE*, [Online]. Available: https://cwe.mitre.org/. [Accessed 27 April 2026].

[3] OWASP Foundation, "OWASP Top 10 Proactive Controls v4.0," OWASP Foundation, Technical Document, 2024. [Online]. Available: https://owasp.org/www-project-proactive-controls/. [Accessed 27 April 2026].

[4] OWASP Foundation, "OWASP Application Security Verification Standard 5.0.0," OWASP Foundation, Technical Standard, 2025. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/. [Accessed 28 April 2026].

[5] PortSwigger, "Burp Suite Community Edition," *PortSwigger*, [Online]. Available: https://portswigger.net/burp/communitydownload. [Accessed 27 April 2026].

[6] OWASP Foundation, "OWASP Top 10:2025," *OWASP*, [Online]. Available: https://owasp.org/Top10/2025/. [Accessed 28 April 2026].

[7] CrackStation, "CrackStation – Online Password Hash Cracking," *CrackStation*, [Online]. Available: https://crackstation.net/. [Accessed 29 April 2026].

[8] MD5Hashing.net, "MD5 Hash Decrypt and Encrypt," *MD5Hashing.net*, [Online]. Available: https://md5hashing.net/. [Accessed 29 April 2026].

[9] Wikipedia, "James T. Kirk," *Wikipedia*, [Online]. Available: https://en.wikipedia.org/wiki/James_T._Kirk. [Accessed 29 April 2026].

[10] totp.app, "TOTP Authenticator," *totp.app*, [Online]. Available: https://totp.app/. [Accessed 29 April 2026].

[11] Semgrep, Inc., "Semgrep," *Semgrep*, [Online]. Available: https://semgrep.dev/. [Accessed 29 April 2026].

[12] OWASP Foundation, "OWASP Juice Shop," *OWASP*, [Online]. Available: https://owasp.org/www-project-juice-shop/. [Accessed 26 April 2026].

[13] OWASP Foundation, "juice-shop [Source code]," *GitHub*, 2014. [Online]. Available: https://github.com/juice-shop/juice-shop. [Accessed 26 April 2026].

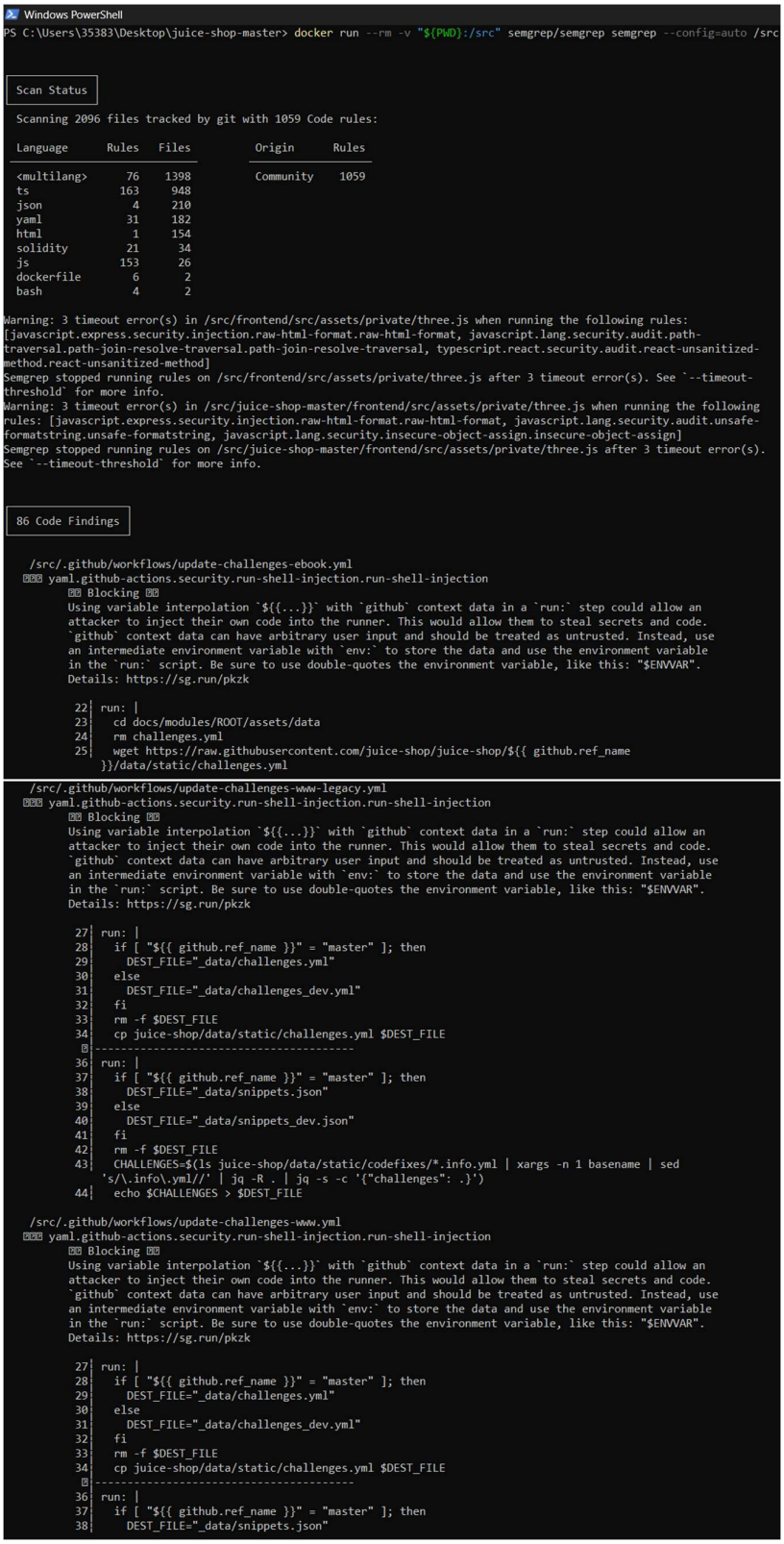
[14] Checkmarx, "ZAP – Zed Attack Proxy," *ZAP by Checkmarx*, [Online]. Available: https://www.zaproxy.org/. [Accessed 26 April 2026].

[15] Mozilla, "Content Security Policy (CSP)," *MDN Web Docs*, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP. [Accessed 26 April 2026].

[16] Mozilla, "X-Frame-Options header," *MDN Web Docs*, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options. [Accessed 26 April 2026].

Appendices

Appendix A: SAST Scan Report – Róisín Aylward



```
Windows PowerShell
PS C:\Users\35383\Desktop\juice-shop-master> docker run --rm -v "${PWD}:/src" semgrep/semgrep semgrep --config=auto /src


  Scan Status

  Scanning 2096 files tracked by git with 1059 Code rules:

  Language      Rules   Files          Origin      Rules
 ─────────────────────────────         ──────────────────
  <multilang>      76    1398          Community    1059
  ts              163     948
  json              4     210
  yaml             31     182
  html              1     154
  solidity         21      34
  js              153      26
  dockerfile        6       2
  bash              4       2

Warning: 3 timeout error(s) in /src/frontend/src/assets/private/three.js when running the following rules:
[javascript.express.security.injection.raw-html-format.raw-html-format, javascript.lang.security.audit.path-
traversal.path-join-resolve-traversal.path-join-resolve-traversal, typescript.react.security.audit.react-unsanitized-
method.react-unsanitized-method]
Semgrep stopped running rules on /src/frontend/src/assets/private/three.js after 3 timeout error(s). See `--timeout-
threshold` for more info.
Warning: 3 timeout error(s) in /src/juice-shop-master/frontend/src/assets/private/three.js when running the following
rules: [javascript.express.security.injection.raw-html-format.raw-html-format, javascript.lang.security.audit.unsafe-
formatstring.unsafe-formatstring, javascript.lang.security.insecure-object-assign.insecure-object-assign]
Semgrep stopped running rules on /src/juice-shop-master/frontend/src/assets/private/three.js after 3 timeout error(s).
See `--timeout-threshold` for more info.


  86 Code Findings


    /src/.github/workflows/update-challenges-ebook.yml
   ❯❯❱ yaml.github-actions.security.run-shell-injection.run-shell-injection
          ❯❯ Blocking ❮❮
          Using variable interpolation `${{...}}` with `github` context data in a `run:` step could allow an
          attacker to inject their own code into the runner. This would allow them to steal secrets and code.
          `github` context data can have arbitrary user input and should be treated as untrusted. Instead, use
          an intermediate environment variable with `env:` to store the data and use the environment variable
          in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".
          Details: https://sg.run/pkzk

           22┆ run: |
           23┆   cd docs/modules/ROOT/assets/data
           24┆   rm challenges.yml
           25┆   wget https://raw.githubusercontent.com/juice-shop/juice-shop/${{ github.ref_name
               }}/data/static/challenges.yml

    /src/.github/workflows/update-challenges-www-legacy.yml
   ❯❯❱ yaml.github-actions.security.run-shell-injection.run-shell-injection
          ❯❯ Blocking ❮❮
          Using variable interpolation `${{...}}` with `github` context data in a `run:` step could allow an
          attacker to inject their own code into the runner. This would allow them to steal secrets and code.
          `github` context data can have arbitrary user input and should be treated as untrusted. Instead, use
          an intermediate environment variable with `env:` to store the data and use the environment variable
          in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".
          Details: https://sg.run/pkzk

           27┆ run: |
           28┆   if [ "${{ github.ref_name }}" = "master" ]; then
           29┆     DEST_FILE="_data/challenges.yml"
           30┆   else
           31┆     DEST_FILE="_data/challenges_dev.yml"
           32┆   fi
           33┆   rm -f $DEST_FILE
           34┆   cp juice-shop/data/static/challenges.yml $DEST_FILE
            ⋮┆----------------------------------------
           36┆ run: |
           37┆   if [ "${{ github.ref_name }}" = "master" ]; then
           38┆     DEST_FILE="_data/snippets.json"
           39┆   else
           40┆     DEST_FILE="_data/snippets_dev.json"
           41┆   fi
           42┆   rm -f $DEST_FILE
           43┆   CHALLENGES=$(ls juice-shop/data/static/codefixes/*.info.yml | xargs -n 1 basename | sed
               's/\.info\.yml//' | jq -R . | jq -s -c '{"challenges": .}')
           44┆   echo $CHALLENGES > $DEST_FILE

    /src/.github/workflows/update-challenges-www.yml
   ❯❯❱ yaml.github-actions.security.run-shell-injection.run-shell-injection
          ❯❯ Blocking ❮❮
          Using variable interpolation `${{...}}` with `github` context data in a `run:` step could allow an
          attacker to inject their own code into the runner. This would allow them to steal secrets and code.
          `github` context data can have arbitrary user input and should be treated as untrusted. Instead, use
          an intermediate environment variable with `env:` to store the data and use the environment variable
          in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".
          Details: https://sg.run/pkzk

           27┆ run: |
           28┆   if [ "${{ github.ref_name }}" = "master" ]; then
           29┆     DEST_FILE="_data/challenges.yml"
           30┆   else
           31┆     DEST_FILE="_data/challenges_dev.yml"
           32┆   fi
           33┆   rm -f $DEST_FILE
           34┆   cp juice-shop/data/static/challenges.yml $DEST_FILE
            ⋮┆----------------------------------------
           36┆ run: |
           37┆   if [ "${{ github.ref_name }}" = "master" ]; then
           38┆     DEST_FILE="_data/snippets.json"
```

```
        39┆   else
        40┆     DEST_FILE="_data/snippets_dev.yml"
        41┆   fi
        42┆   rm -f $DEST_FILE
        43┆   CHALLENGES=$(ls juice-shop/data/static/codefixes/*.info.yml | xargs -n 1 basename | sed
          's/\.info\.yml//' | jq -R . | jq -s -c '{"challenges": .}')
        44┆   echo $CHALLENGES > $DEST_FILE

 /src/data/static/codefixes/dbSchemaChallenge_1.ts
❯❯❱ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❯❯
       Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
       the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
       is recommended to use parameterized queries or prepared statements.
       Details: https://sg.run/gjoe

         5┆ models.sequelize.query("SELECT * FROM Products WHERE ((name LIKE '%"+criteria+"%' OR
           description LIKE '%"+criteria+"%') AND deletedAt IS NULL) ORDER BY name")

 /src/data/static/codefixes/dbSchemaChallenge_3.ts
❯❯❱ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❯❯
       Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
       the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
       is recommended to use parameterized queries or prepared statements.
       Details: https://sg.run/gjoe

        11┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
           description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)

 /src/data/static/codefixes/restfulXssChallenge_2.ts
  ❯ javascript.audit.detect-replaceall-sanitization.detect-replaceall-sanitization
       ❯❯ Blocking ❯❯
       Detected a call to `replaceAll()` in an attempt to HTML escape the string
       `tableData[i].description`. Manually sanitizing input through a manually built list can be
       circumvented in many situations, and it's better to use a well known sanitization library such as
       `sanitize-html` or `DOMPurify`.
       Details: https://sg.run/AzoB

        49┆ tableData[i].description = tableData[i].description.replaceAll('<', '&lt;').replaceAll('>',
           '&gt;')
         ⋮┆----------------------------------------
  ❯ javascript.audit.detect-replaceall-sanitization.detect-replaceall-sanitization
       ❯❯ Blocking ❯❯
       Detected a call to `replaceAll()` in an attempt to HTML escape the string
       `tableData[i].description.replaceAll('<', '&lt;')`. Manually sanitizing input through a manually
       built list can be circumvented in many situations, and it's better to use a well known sanitization
       library such as `sanitize-html` or `DOMPurify`.
       Details: https://sg.run/AzoB

        49┆ tableData[i].description = tableData[i].description.replaceAll('<', '&lt;').replaceAll('>',
           '&gt;')

 /src/data/static/codefixes/unionSqlInjectionChallenge_1.ts
❯❯❱ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❯❯
       Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
       the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
       is recommended to use parameterized queries or prepared statements.
       Details: https://sg.run/gjoe

         6┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
           description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)

 /src/data/static/codefixes/unionSqlInjectionChallenge_3.ts
❯❯❱ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❯❯
       Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
       the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
       is recommended to use parameterized queries or prepared statements.
       Details: https://sg.run/gjoe

        10┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
           description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)

 /src/data/static/users.yml
❯❯❱ generic.secrets.security.detected-generic-secret.detected-generic-secret
       ❯❯ Blocking ❯❯
       Generic Secret detected
       Details: https://sg.run/l2o5

       151┆ totpSecret: IFTXE3SPOEYVURT2MRYGI52TKJ4HC3KH

 /src/frontend/src/app/app.guard.spec.ts
❯❯❱ generic.secrets.security.detected-jwt-token.detected-jwt-token
       ❯❯ Blocking ❯❯
       JWT token detected
       Details: https://sg.run/05N5

        38┆ localStorage.setItem('token', 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODk
           wIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQs
           sw5c')

 /src/frontend/src/app/last-login-ip/last-login-ip.component.spec.ts
❯❯❱ generic.secrets.security.detected-jwt-token.detected-jwt-token
       ❯❯ Blocking ❯❯
       JWT token detected
       Details: https://sg.run/05N5

        61┆ localStorage.setItem('token', 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJkYXRhIjp7Imxhc3RMb2d
           pbklwIjoiMS4yLjMuNCJ9fQ.RAkmdqwNypuOxv3SDjPO4xMKvd1CddKvDFYDBfUt3bg')
         ⋮┆----------------------------------------
        67┆ localStorage.setItem('token', 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJkYXRhIjp7fX0.bVBhvll
           6IaeR3aUdoOeyR8YZe2S2DfhGAxTGfd9enLw')
```

```
 /src/frontend/src/hacking-instructor/helpers/helpers.ts
❯❯ javascript.lang.security.audit.prototype-pollution.prototype-pollution-loop.prototype-pollution-loop
       ❯❯ Blocking ❯❯
       Possibility of prototype polluting function detected. By adding or modifying attributes of an object
       prototype, it is possible to create attributes that exist on every object, or replace critical
       attributes with malicious ones. This can be problematic if the software depends on existence or non-
       existence of certain attributes, or uses pre-defined attributes of object prototype (such as
       hasOwnProperty, toString or valueOf). Possible mitigations might be: freezing the object prototype,
       using an object without prototypes (via Object.create(null) ), blocking modifications of attributes
       that resolve to object prototype, using Map instead of object.
       Details: https://sg.run/w1DB

        49┆ replacementValue = replacementValue[property]

 /src/juice-shop-master/.github/workflows/update-challenges-ebook.yml
❯❯❯ yaml.github-actions.security.run-shell-injection.run-shell-injection
       ❯❯ Blocking ❯❯
       Using variable interpolation `${{...}}` with `github` context data in a `run:` step could allow an
       attacker to inject their own code into the runner. This would allow them to steal secrets and code.
       `github` context data can have arbitrary user input and should be treated as untrusted. Instead, use
       an intermediate environment variable with `env:` to store the data and use the environment variable
       in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".
       Details: https://sg.run/pkzk

        22┆ run: |
        23┆   cd docs/modules/ROOT/assets/data
        24┆   rm challenges.yml
        25┆   wget https://raw.githubusercontent.com/juice-shop/juice-shop/${{ github.ref_name
           }}/data/static/challenges.yml

 /src/juice-shop-master/.github/workflows/update-challenges-www-legacy.yml
❯❯❯ yaml.github-actions.security.run-shell-injection.run-shell-injection
       ❯❯ Blocking ❯❯
       Using variable interpolation `${{...}}` with `github` context data in a `run:` step could allow an
       attacker to inject their own code into the runner. This would allow them to steal secrets and code.
       `github` context data can have arbitrary user input and should be treated as untrusted. Instead, use
       an intermediate environment variable with `env:` to store the data and use the environment variable
       in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".
       Details: https://sg.run/pkzk

        27┆ run: |
        28┆   if [ "${{ github.ref_name }}" = "master" ]; then
        29┆     DEST_FILE="_data/challenges.yml"
        30┆   else
        31┆     DEST_FILE="_data/challenges_dev.yml"
        32┆   fi
        33┆   rm -f $DEST_FILE
        34┆   cp juice-shop/data/static/challenges.yml $DEST_FILE
         ⋮┆----------------------------------------
        36┆ run: |
        37┆   if [ "${{ github.ref_name }}" = "master" ]; then
        38┆     DEST_FILE="_data/snippets.json"
        39┆   else
        40┆     DEST_FILE="_data/snippets_dev.json"
        41┆   fi
        42┆   rm -f $DEST_FILE
        43┆   CHALLENGES=$(ls juice-shop/data/static/codefixes/*.info.yml | xargs -n 1 basename | sed
           's/\.info\.yml//' | jq -R . | jq -s -c '{"challenges": .}')
        44┆   echo $CHALLENGES > $DEST_FILE

 /src/juice-shop-master/.github/workflows/update-challenges-www.yml
❯❯❯ yaml.github-actions.security.run-shell-injection.run-shell-injection
       ❯❯ Blocking ❯❯
       Using variable interpolation `${{...}}` with `github` context data in a `run:` step could allow an
       attacker to inject their own code into the runner. This would allow them to steal secrets and code.
       `github` context data can have arbitrary user input and should be treated as untrusted. Instead, use
       an intermediate environment variable with `env:` to store the data and use the environment variable
       in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".
       Details: https://sg.run/pkzk

        27┆ run: |
        28┆   if [ "${{ github.ref_name }}" = "master" ]; then
        29┆     DEST_FILE="_data/challenges.yml"
        30┆   else
        31┆     DEST_FILE="_data/challenges_dev.yml"
        32┆   fi
        33┆   rm -f $DEST_FILE
        34┆   cp juice-shop/data/static/challenges.yml $DEST_FILE
         ⋮┆----------------------------------------
        36┆ run: |
        37┆   if [ "${{ github.ref_name }}" = "master" ]; then
        38┆     DEST_FILE="_data/snippets.json"
        39┆   else
        40┆     DEST_FILE="_data/snippets_dev.yml"
        41┆   fi
        42┆   rm -f $DEST_FILE
        43┆   CHALLENGES=$(ls juice-shop/data/static/codefixes/*.info.yml | xargs -n 1 basename | sed
           's/\.info\.yml//' | jq -R . | jq -s -c '{"challenges": .}')
        44┆   echo $CHALLENGES > $DEST_FILE

 /src/juice-shop-master/data/static/codefixes/dbSchemaChallenge_1.ts
❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❯❯
       Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
       the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
       is recommended to use parameterized queries or prepared statements.
       Details: https://sg.run/gjoe

         5┆ models.sequelize.query("SELECT * FROM Products WHERE ((name LIKE '%"+criteria+"%' OR
           description LIKE '%"+criteria+"%') AND deletedAt IS NULL) ORDER BY name")

 /src/juice-shop-master/data/static/codefixes/dbSchemaChallenge_3.ts
❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❯❯
```

```
        Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
        the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
        is recommended to use parameterized queries or prepared statements.
        Details: https://sg.run/gjoe

         11┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
             description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)

  /src/juice-shop-master/data/static/codefixes/restfulXssChallenge_2.ts
   ❯ javascript.audit.detect-replaceall-sanitization.detect-replaceall-sanitization
        ❯❯ Blocking ❯❯
        Detected a call to `replaceAll()` in an attempt to HTML escape the string
        `tableData[i].description`. Manually sanitizing input through a manually built list can be
        circumvented in many situations, and it's better to use a well known sanitization library such as
        `sanitize-html` or `DOMPurify`.
        Details: https://sg.run/AzoB

         49┆ tableData[i].description = tableData[i].description.replaceAll('<', '&lt;').replaceAll('>',
             '&gt;')
          ⋮┆----------------------------------------
   ❯ javascript.audit.detect-replaceall-sanitization.detect-replaceall-sanitization
        ❯❯ Blocking ❯❯
        Detected a call to `replaceAll()` in an attempt to HTML escape the string
        `tableData[i].description.replaceAll('<', '&lt;')`. Manually sanitizing input through a manually
        built list can be circumvented in many situations, and it's better to use a well known sanitization
        library such as `sanitize-html` or `DOMPurify`.
        Details: https://sg.run/AzoB

         49┆ tableData[i].description = tableData[i].description.replaceAll('<', '&lt;').replaceAll('>',
             '&gt;')

  /src/juice-shop-master/data/static/codefixes/unionSqlInjectionChallenge_1.ts
❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
        ❯❯ Blocking ❯❯
        Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
        the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
        is recommended to use parameterized queries or prepared statements.
        Details: https://sg.run/gjoe

          6┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
             description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)

  /src/juice-shop-master/data/static/codefixes/unionSqlInjectionChallenge_3.ts
❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
        ❯❯ Blocking ❯❯
        Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
        the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
        is recommended to use parameterized queries or prepared statements.
        Details: https://sg.run/gjoe

         10┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
             description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)
```

```
  /src/juice-shop-master/data/static/users.yml
❯❯❯ generic.secrets.security.detected-generic-secret.detected-generic-secret
        ❯❯ Blocking ❯❯
        Generic Secret detected
        Details: https://sg.run/l2o5

        151┆ totpSecret: IFTXE3SPOEYVURT2MRYGI52TKJ4HC3KH

  /src/juice-shop-master/frontend/src/app/app.guard.spec.ts
❯❯❯ generic.secrets.security.detected-jwt-token.detected-jwt-token
        ❯❯ Blocking ❯❯
        JWT token detected
        Details: https://sg.run/05N5

         38┆ localStorage.setItem('token', 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODk
             wIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQs
             sw5c')

  /src/juice-shop-master/frontend/src/app/last-login-ip/last-login-ip.component.spec.ts
❯❯❯ generic.secrets.security.detected-jwt-token.detected-jwt-token
        ❯❯ Blocking ❯❯
        JWT token detected
        Details: https://sg.run/05N5

         61┆ localStorage.setItem('token', 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJkYXRhIjp7Imxhc3RMb2d
             pbklwIjoiMS4yLjMuNCJ9fQ.RAkmdqwNypuOxv3SDjPO4xMKvd1CddKvDFYDBfUt3bg')
          ⋮┆----------------------------------------
         67┆ localStorage.setItem('token', 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJkYXRhIjp7fX0.bVBhvll
             6IaeR3aUdoOeyR8YZe2S2DfhGAxTGfd9enLw')

  /src/juice-shop-master/frontend/src/hacking-instructor/helpers/helpers.ts
 ❯❯ javascript.lang.security.audit.prototype-pollution.prototype-pollution-loop.prototype-pollution-loop
        ❯❯ Blocking ❯❯
        Possibility of prototype polluting function detected. By adding or modifying attributes of an object
        prototype, it is possible to create attributes that exist on every object, or replace critical
        attributes with malicious ones. This can be problematic if the software depends on existence or non-
        existence of certain attributes, or uses pre-defined attributes of object prototype (such as
        hasOwnProperty, toString or valueOf). Possible mitigations might be: freezing the object prototype,
        using an object without prototypes (via Object.create(null) ), blocking modifications of attributes
        that resolve to object prototype, using Map instead of object.
        Details: https://sg.run/w1DB

         49┆ replacementValue = replacementValue[property]

  /src/juice-shop-master/lib/codingChallenges.ts
 ❯❯ javascript.lang.security.audit.detect-non-literal-regexp.detect-non-literal-regexp
        ❯❯ Blocking ❯❯
        RegExp() called with a `challengeKey` function argument, this might allow an attacker to cause a
        Regular Expression Denial-of-Service (ReDoS) within your application as RegExP blocks the main
        thread. For this reason, it is recommended to use hardcoded regexes instead. If your regex is run on
        user-controlled input, consider performing input validation or use a regex checking/sanitization
        library such as https://www.npmjs.com/package/recheck to verify that the regex does not appear
```

```
       vulnerable to ReDoS.
       Details: https://sg.run/gr65

        76┆ if (new RegExp(`vuln-code-snippet vuln-line.*${challengeKey}`).exec(lines[i]) != null) {
         ⋮┆----------------------------------------
        78┆ } else if (new RegExp(`vuln-code-snippet neutral-line.*${challengeKey}`).exec(lines[i]) !=
            null) {

  /src/juice-shop-master/lib/insecurity.ts
  ❯❯ javascript.lang.security.audit.hardcoded-hmac-key.hardcoded-hmac-key
       ❯❯ Blocking ❯❯
       Detected a hardcoded hmac key. Avoid hardcoding secrets and consider using an alternate option such
       as reading the secret from a config file or using an environment variable.
       Details: https://sg.run/K9bn

        44┆ export const hmac = (data: string) => crypto.createHmac('sha256',
            'pa4qacea4VK9t9nGv7yZtwmj').update(data).digest('hex')

  ❯❯ javascript.jsonwebtoken.security.jwt-hardcode.hardcoded-jwt-secret
       ❯❯ Blocking ❯❯
       A hard-coded credential was detected. It is not recommended to store credentials in source-code, as
       this risks secrets being leaked and used by either an internal or external malicious adversary. It
       is recommended to use environment variables to securely provide credentials or retrieve credentials
       from a secure vault or HSM (Hardware Security Module).
       Details: https://sg.run/4xN9

        56┆ export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h',
            algorithm: 'RS256' })

  ❯❯ javascript.lang.security.audit.hardcoded-hmac-key.hardcoded-hmac-key
       ❯❯ Blocking ❯❯
       Detected a hardcoded hmac key. Avoid hardcoding secrets and consider using an alternate option such
       as reading the secret from a config file or using an environment variable.
       Details: https://sg.run/K9bn

       152┆ const hmac = crypto.createHmac('sha256', privateKey)

  /src/juice-shop-master/routes/b2bOrder.ts
  ❯❯ javascript.express.security.audit.express-detect-notevil-usage.express-detect-notevil-usage
       ❯❯ Blocking ❯❯
       Detected usage of the `notevil` package, which is unmaintained and has vulnerabilities. Using any
       sort of `eval()` functionality can be very dangerous, but if you must, the `eval` package is an up
       to date alternative. Be sure that only trusted input reaches an `eval()` function.
       Details: https://sg.run/W70E

        23┆ vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })

  /src/juice-shop-master/routes/captcha.ts
  ❯❯ javascript.browser.security.eval-detected.eval-detected
       ❯❯ Blocking ❯❯
       Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
       content can be input from outside the program, this may be a code injection vulnerability. Ensure
       evaluated content is not definable by external sources.
       Details: https://sg.run/7ope

        22┆ const answer = eval(expression).toString() // eslint-disable-line no-eval

  /src/juice-shop-master/routes/chatbot.ts
  ❯❯ javascript.express.security.injection.raw-html-format.raw-html-format
       ❯❯ Blocking ❯❯
       User data flows into the host portion of this manually-constructed HTML. This can introduce a Cross-
       Site-Scripting (XSS) vulnerability if this comes from user-provided input. Consider using a
       sanitization library such as DOMPurify to sanitize the HTML within.
       Details: https://sg.run/5DO3

       205┆ body: bot.training.state ? bot.greet(`${user.id}`) :
            `${config.get<string>('application.chatBot.name')} isn't ready at the moment, please wait
            while I set things up`

  /src/juice-shop-master/routes/currentUser.ts
  ❯❯❯ javascript.express.security.audit.remote-property-injection.remote-property-injection
       ❯❯ Blocking ❯❯
       Bracket object notation with user input is present, this might allow an attacker to access all
       properties of the object and even it's prototype. Use literal values for object properties.
       Details: https://sg.run/Z4gn

        31┆ baseUser[field] = user?.data[field as keyof typeof user.data]

  /src/juice-shop-master/routes/fileServer.ts
  ❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
       ❯❯ Blocking ❯❯
       The application processes user-input, this is passed to res.sendFile which can allow an attacker to
       arbitrarily read files on the system through path traversal. It is recommended to perform input
       validation in addition to canonicalizing the path. This allows you to validate the path against the
       intended directory it should be accessing.
       Details: https://sg.run/7DJk

        33┆ res.sendFile(path.resolve('ftp/', file))

  /src/juice-shop-master/routes/fileUpload.ts
  ❯❯ javascript.express.security.audit.express-libxml-vm-noent.express-libxml-vm-noent
       ❯❯ Blocking ❯❯
       Detected use of parseXml() function with the `noent` field set to `true`. This can lead to an XML
       External Entities (XXE) attack if untrusted data is passed into it.
       Details: https://sg.run/n8Ag

        83┆ const xmlDoc = vm.runInContext('libxml.parseXml(data, { noblanks: true, noent: true,
            nocdata: true })', sandbox, { timeout: 2000 })

  /src/juice-shop-master/routes/keyServer.ts
  ❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
       ❯❯ Blocking ❯❯
       The application processes user-input, this is passed to res.sendFile which can allow an attacker to
       arbitrarily read files on the system through path traversal. It is recommended to perform input
```

```
        validation in addition to canonicalizing the path. This allows you to validate the path against the
        intended directory it should be accessing.
        Details: https://sg.run/7DJk

         14┆ res.sendFile(path.resolve('encryptionkeys/', file))

  /src/juice-shop-master/routes/logfileServer.ts
  ❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
        ❯❯ Blocking ❯❯
        The application processes user-input, this is passed to res.sendFile which can allow an attacker to
        arbitrarily read files on the system through path traversal. It is recommended to perform input
        validation in addition to canonicalizing the path. This allows you to validate the path against the
        intended directory it should be accessing.
        Details: https://sg.run/7DJk

         14┆ res.sendFile(path.resolve('logs/', file))

  /src/juice-shop-master/routes/login.ts
  ❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
        ❯❯ Blocking ❯❯
        Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
        the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
        is recommended to use parameterized queries or prepared statements.
        Details: https://sg.run/gjoe

         34┆ models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND
             password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
             UserModel, plain: true }) // vuln-code-snippet vuln-line loginAdminChallenge
             loginBenderChallenge loginJimChallenge

  /src/juice-shop-master/routes/quarantineServer.ts
  ❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
        ❯❯ Blocking ❯❯
        The application processes user-input, this is passed to res.sendFile which can allow an attacker to
        arbitrarily read files on the system through path traversal. It is recommended to perform input
        validation in addition to canonicalizing the path. This allows you to validate the path against the
        intended directory it should be accessing.
        Details: https://sg.run/7DJk

         14┆ res.sendFile(path.resolve('ftp/quarantine/', file))

  /src/juice-shop-master/routes/redirect.ts
  ❯❯ javascript.express.security.audit.express-open-redirect.express-open-redirect
        ❯❯ Blocking ❯❯
        The application redirects to a URL specified by user-supplied input `query` that is not validated.
        This could redirect users to malicious locations. Consider using an allow-list approach to validate
        URLs, or warn users they are being redirected to a third-party website.
        Details: https://sg.run/EpoP

         19┆ res.redirect(toUrl)

  /src/juice-shop-master/routes/search.ts
  ❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
        ❯❯ Blocking ❯❯
        Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
        the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
        is recommended to use parameterized queries or prepared statements.
        Details: https://sg.run/gjoe

         23┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
             description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`) // vuln-code-
             snippet vuln-line unionSqlInjectionChallenge dbSchemaChallenge

  /src/juice-shop-master/routes/userProfile.ts
  ❯❯ javascript.browser.security.eval-detected.eval-detected
        ❯❯ Blocking ❯❯
        Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
        content can be input from outside the program, this may be a code injection vulnerability. Ensure
        evaluated content is not definable by external sources.
        Details: https://sg.run/7ope

         62┆ username = eval(code) // eslint-disable-line no-eval

  ❯❯❯ javascript.lang.security.audit.code-string-concat.code-string-concat
        ❯❯ Blocking ❯❯
        Found data from an Express or Next web request flowing to `eval`. If this data is user-controllable
        this can lead to execution of arbitrary system commands in the context of your application process.
        Avoid `eval` whenever possible.
        Details: https://sg.run/96Yk

         62┆ username = eval(code) // eslint-disable-line no-eval

  /src/juice-shop-master/routes/videoHandler.ts
  ❯❯ javascript.lang.security.audit.unknown-value-with-script-tag.unknown-value-with-script-tag
        ❯❯ Blocking ❯❯
        Cannot determine what 'subs' is and it is used with a '<script>' tag. This could be susceptible to
        cross-site scripting (XSS). Ensure 'subs' is not externally controlled, or sanitize this data.
        Details: https://sg.run/1Zy1

         58┆ challengeUtils.solveIf(challenges.videoXssChallenge, () => { return utils.contains(subs,
             '</script><script>alert(`xss`)</script>') })
           ⋮┆----------------------------------------
         71┆ compiledTemplate = compiledTemplate.replace('<script id="subtitle"></script>', '<script
             id="subtitle" type="text/vtt" data-label="English" data-lang="en">' + subs + '</script>')

  /src/juice-shop-master/server.ts
    ❯ javascript.lang.security.audit.unsafe-formatstring.unsafe-formatstring
        ❯❯ Blocking ❯❯
        Detected string concatenation with a non-literal variable in a util.format / console.log function.
        If an attacker injects a format specifier in the string, it will forge the log message. Try to use
        constant values for the format string.
        Details: https://sg.run/7Y5R

        155┆ console.error('Error in timed startup function: ' + name, err)
```

```
❯❯ javascript.express.security.audit.express-check-directory-listing.express-check-directory-listing
      ❯❯ Blocking ❯❯
      Directory listing/indexing is enabled, which may lead to disclosure of sensitive directories and
      files. It is recommended to disable directory listing unless it is a public resource. If you need
      directory listing, ensure that sensitive files are inaccessible when querying the resource.
      Details: https://sg.run/DX2G

      269┆ app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true })) // vuln-code-
          snippet vuln-line directoryListingChallenge
        ⋮┆----------------------------------------
      273┆ app.use('/.well-known', serveIndexMiddleware, serveIndex('.well-known', { icons: true,
          view: 'details' }))
        ⋮┆----------------------------------------
      277┆ app.use('/encryptionkeys', serveIndexMiddleware, serveIndex('encryptionkeys', { icons:
          true, view: 'details' }))
        ⋮┆----------------------------------------
      281┆ app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view:
          'details' })) // vuln-code-snippet vuln-line accessLogDisclosureChallenge

  /src/juice-shop-master/views/promotionVideo.pug
  ❯❯ javascript.express.security.audit.xss.pug.explicit-unescape.template-explicit-unescape
      ❯❯ Blocking ❯❯
      Detected an explicit unescape in a Pug template, using either '!=' or '!{...}'. If external data can
      reach these locations, your application is exposed to a cross-site scripting (XSS) vulnerability. If
      you must do this, ensure no external data can reach this location.
      Details: https://sg.run/3xbe

       78┆ if (splitted.length != 2) {

  /src/lib/codingChallenges.ts
  ❯❯ javascript.lang.security.audit.detect-non-literal-regexp.detect-non-literal-regexp
      ❯❯ Blocking ❯❯
      RegExp() called with a `challengeKey` function argument, this might allow an attacker to cause a
      Regular Expression Denial-of-Service (ReDoS) within your application as RegExP blocks the main
      thread. For this reason, it is recommended to use hardcoded regexes instead. If your regex is run on
      user-controlled input, consider performing input validation or use a regex checking/sanitization
      library such as https://www.npmjs.com/package/recheck to verify that the regex does not appear
      vulnerable to ReDoS.
      Details: https://sg.run/gr65

       76┆ if (new RegExp(`vuln-code-snippet vuln-line.*${challengeKey}`).exec(lines[i]) != null) {
        ⋮┆----------------------------------------
       78┆ } else if (new RegExp(`vuln-code-snippet neutral-line.*${challengeKey}`).exec(lines[i]) !=
          null) {

  /src/lib/insecurity.ts
  ❯❯ javascript.lang.security.audit.hardcoded-hmac-key.hardcoded-hmac-key
      ❯❯ Blocking ❯❯
      Detected a hardcoded hmac key. Avoid hardcoding secrets and consider using an alternate option such
      as reading the secret from a config file or using an environment variable.
      Details: https://sg.run/K9bn

       44┆ export const hmac = (data: string) => crypto.createHmac('sha256',
          'pa4qacea4VK9t9nGv7yZtwmj').update(data).digest('hex')

  ❯❯ javascript.jsonwebtoken.security.jwt-hardcode.hardcoded-jwt-secret
      ❯❯ Blocking ❯❯
      A hard-coded credential was detected. It is not recommended to store credentials in source-code, as
      this risks secrets being leaked and used by either an internal or external malicious adversary. It
      is recommended to use environment variables to securely provide credentials or retrieve credentials
      from a secure vault or HSM (Hardware Security Module).
      Details: https://sg.run/4xN9

       56┆ export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h',
          algorithm: 'RS256' })

  ❯❯ javascript.lang.security.audit.hardcoded-hmac-key.hardcoded-hmac-key
      ❯❯ Blocking ❯❯
      Detected a hardcoded hmac key. Avoid hardcoding secrets and consider using an alternate option such
      as reading the secret from a config file or using an environment variable.
      Details: https://sg.run/K9bn

      152┆ const hmac = crypto.createHmac('sha256', privateKey)

  /src/routes/b2bOrder.ts
  ❯❯ javascript.express.security.audit.express-detect-notevil-usage.express-detect-notevil-usage
      ❯❯ Blocking ❯❯
      Detected usage of the `notevil` package, which is unmaintained and has vulnerabilities. Using any
      sort of `eval()` functionality can be very dangerous, but if you must, the `eval` package is an up
      to date alternative. Be sure that only trusted input reaches an `eval()` function.
      Details: https://sg.run/W70E

       23┆ vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })

  /src/routes/captcha.ts
  ❯❯ javascript.browser.security.eval-detected.eval-detected
      ❯❯ Blocking ❯❯
      Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
      content can be input from outside the program, this may be a code injection vulnerability. Ensure
      evaluated content is not definable by external sources.
      Details: https://sg.run/7ope

       22┆ const answer = eval(expression).toString() // eslint-disable-line no-eval

  /src/routes/chatbot.ts
  ❯❯ javascript.express.security.injection.raw-html-format.raw-html-format
      ❯❯ Blocking ❯❯
      User data flows into the host portion of this manually-constructed HTML. This can introduce a Cross-
      Site-Scripting (XSS) vulnerability if this comes from user-provided input. Consider using a
      sanitization library such as DOMPurify to sanitize the HTML within.
      Details: https://sg.run/5DO3

      205┆ body: bot.training.state ? bot.greet(`${user.id}`) :
          `${config.get<string>('application.chatBot.name')} isn't ready at the moment, please wait
```

```
            while I set things up`

  /src/routes/currentUser.ts
❯❯❯ javascript.express.security.audit.remote-property-injection.remote-property-injection
       ❯❯ Blocking ❮❮
       Bracket object notation with user input is present, this might allow an attacker to access all
       properties of the object and even it's prototype. Use literal values for object properties.
       Details: https://sg.run/Z4gn

        31┆ baseUser[field] = user?.data[field as keyof typeof user.data]

  /src/routes/fileServer.ts
❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
       ❯❯ Blocking ❮❮
       The application processes user-input, this is passed to res.sendFile which can allow an attacker to
       arbitrarily read files on the system through path traversal. It is recommended to perform input
       validation in addition to canonicalizing the path. This allows you to validate the path against the
       intended directory it should be accessing.
       Details: https://sg.run/7DJk

        33┆ res.sendFile(path.resolve('ftp/', file))

  /src/routes/fileUpload.ts
❯❯ javascript.express.security.audit.express-libxml-vm-noent.express-libxml-vm-noent
       ❯❯ Blocking ❮❮
       Detected use of parseXml() function with the `noent` field set to `true`. This can lead to an XML
       External Entities (XXE) attack if untrusted data is passed into it.
       Details: https://sg.run/n8Ag

        83┆ const xmlDoc = vm.runInContext('libxml.parseXml(data, { noblanks: true, noent: true,
            nocdata: true })', sandbox, { timeout: 2000 })

  /src/routes/keyServer.ts
❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
       ❯❯ Blocking ❮❮
       The application processes user-input, this is passed to res.sendFile which can allow an attacker to
       arbitrarily read files on the system through path traversal. It is recommended to perform input
       validation in addition to canonicalizing the path. This allows you to validate the path against the
       intended directory it should be accessing.
       Details: https://sg.run/7DJk

        14┆ res.sendFile(path.resolve('encryptionkeys/', file))

  /src/routes/logfileServer.ts
❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
       ❯❯ Blocking ❮❮
       The application processes user-input, this is passed to res.sendFile which can allow an attacker to
       arbitrarily read files on the system through path traversal. It is recommended to perform input
       validation in addition to canonicalizing the path. This allows you to validate the path against the
       intended directory it should be accessing.
       Details: https://sg.run/7DJk

        14┆ res.sendFile(path.resolve('logs/', file))

  /src/routes/login.ts
❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❮❮
       Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
       the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
       is recommended to use parameterized queries or prepared statements.
       Details: https://sg.run/gjoe

        34┆ models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND
            password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
            UserModel, plain: true }) // vuln-code-snippet vuln-line loginAdminChallenge
            loginBenderChallenge loginJimChallenge

  /src/routes/quarantineServer.ts
❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile
       ❯❯ Blocking ❮❮
       The application processes user-input, this is passed to res.sendFile which can allow an attacker to
       arbitrarily read files on the system through path traversal. It is recommended to perform input
       validation in addition to canonicalizing the path. This allows you to validate the path against the
       intended directory it should be accessing.
       Details: https://sg.run/7DJk

        14┆ res.sendFile(path.resolve('ftp/quarantine/', file))

  /src/routes/redirect.ts
❯❯ javascript.express.security.audit.express-open-redirect.express-open-redirect
       ❯❯ Blocking ❮❮
       The application redirects to a URL specified by user-supplied input `query` that is not validated.
       This could redirect users to malicious locations. Consider using an allow-list approach to validate
       URLs, or warn users they are being redirected to a third-party website.
       Details: https://sg.run/EpoP

        19┆ res.redirect(toUrl)

  /src/routes/search.ts
❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
       ❯❯ Blocking ❮❮
       Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
       the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
       is recommended to use parameterized queries or prepared statements.
       Details: https://sg.run/gjoe

        23┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
            description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`) // vuln-code-
            snippet vuln-line unionSqlInjectionChallenge dbSchemaChallenge

  /src/routes/userProfile.ts
❯❯ javascript.browser.security.eval-detected.eval-detected
       ❯❯ Blocking ❮❮
       Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
```

```
        content can be input from outside the program, this may be a code injection vulnerability. Ensure
        evaluated content is not definable by external sources.
        Details: https://sg.run/7ope

         62┆ username = eval(code) // eslint-disable-line no-eval

  ❯❯❯ javascript.lang.security.audit.code-string-concat.code-string-concat
        ❯❯ Blocking ❮❮
        Found data from an Express or Next web request flowing to `eval`. If this data is user-controllable
        this can lead to execution of arbitrary system commands in the context of your application process.
        Avoid `eval` whenever possible.
        Details: https://sg.run/96Yk

         62┆ username = eval(code) // eslint-disable-line no-eval

   /src/routes/videoHandler.ts
   ❯❯ javascript.lang.security.audit.unknown-value-with-script-tag.unknown-value-with-script-tag
        ❯❯ Blocking ❮❮
        Cannot determine what 'subs' is and it is used with a '<script>' tag. This could be susceptible to
        cross-site scripting (XSS). Ensure 'subs' is not externally controlled, or sanitize this data.
        Details: https://sg.run/1Zy1

         58┆ challengeUtils.solveIf(challenges.videoXssChallenge, () => { return utils.contains(subs,
             '</script><script>alert(`xss`)</script>') })
           ⋮┆----------------------------------------
         71┆ compiledTemplate = compiledTemplate.replace('<script id="subtitle"></script>', '<script
             id="subtitle" type="text/vtt" data-label="English" data-lang="en">' + subs + '</script>')

   /src/server.ts
    ❯ javascript.lang.security.audit.unsafe-formatstring.unsafe-formatstring
        ❯❯ Blocking ❮❮
        Detected string concatenation with a non-literal variable in a util.format / console.log function.
        If an attacker injects a format specifier in the string, it will forge the log message. Try to use
        constant values for the format string.
        Details: https://sg.run/7Y5R

        155┆ console.error('Error in timed startup function: ' + name, err)

   ❯❯ javascript.express.security.audit.express-check-directory-listing.express-check-directory-listing
        ❯❯ Blocking ❮❮
        Directory listing/indexing is enabled, which may lead to disclosure of sensitive directories and
        files. It is recommended to disable directory listing unless it is a public resource. If you need
        directory listing, ensure that sensitive files are inaccessible when querying the resource.
        Details: https://sg.run/DX2G

        269┆ app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true })) // vuln-code-
             snippet vuln-line directoryListingChallenge
           ⋮┆----------------------------------------
        273┆ app.use('/.well-known', serveIndexMiddleware, serveIndex('.well-known', { icons: true,
             view: 'details' }))
           ⋮┆----------------------------------------
        277┆ app.use('/encryptionkeys', serveIndexMiddleware, serveIndex('encryptionkeys', { icons:
             true, view: 'details' }))
           ⋮┆----------------------------------------
        281┆ app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view:
             'details' })) // vuln-code-snippet vuln-line accessLogDisclosureChallenge

   /src/views/promotionVideo.pug
   ❯❯ javascript.express.security.audit.xss.pug.explicit-unescape.template-explicit-unescape
        ❯❯ Blocking ❮❮
        Detected an explicit unescape in a Pug template, using either '!=' or '!{...}'. If external data can
        reach these locations, your application is exposed to a cross-site scripting (XSS) vulnerability. If
        you must do this, ensure no external data can reach this location.
        Details: https://sg.run/3xbe

         78┆ if (splitted.length != 2) {

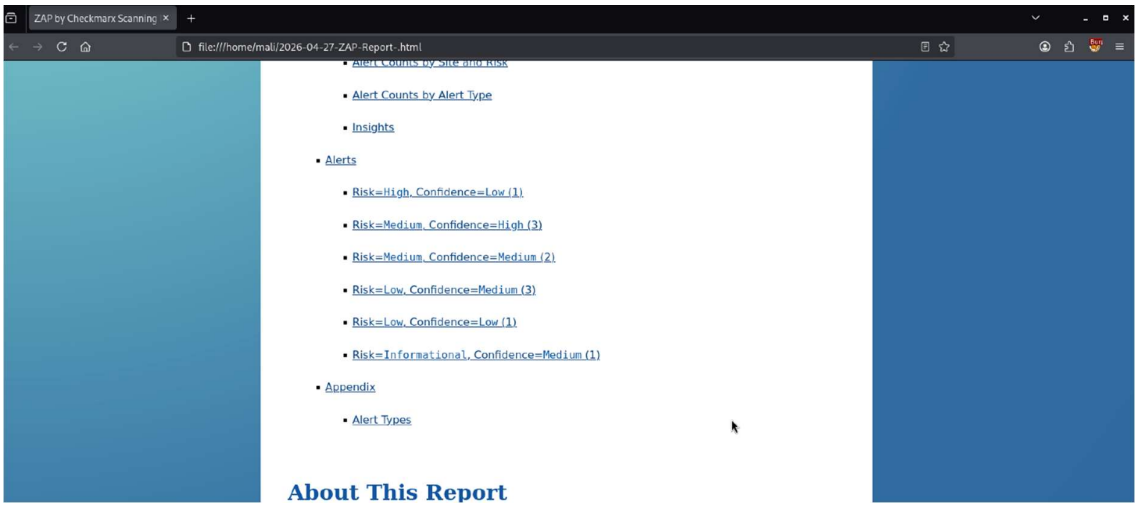

 Scan Summary

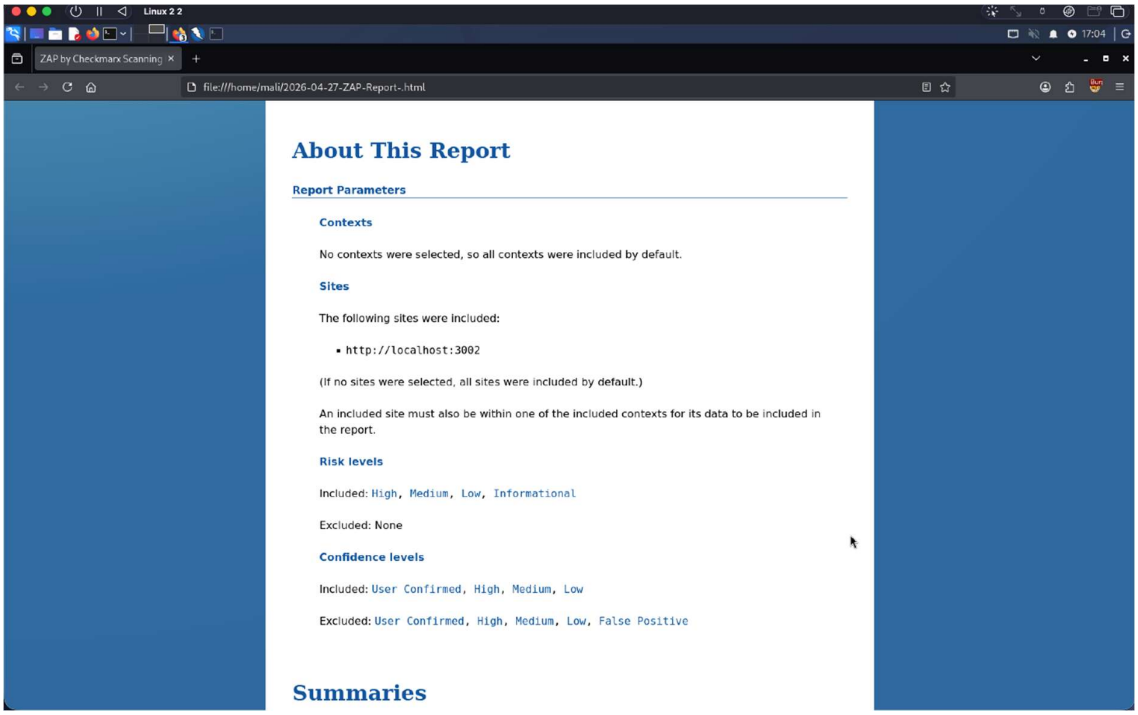
✓ Scan completed successfully.
 • Findings: 86 (86 blocking)
 • Rules run: 302
 • Targets scanned: 2096
 • Parsed lines: ~99.9%
 • Scan skipped:
   ◦ Files larger than  files 1.0 MB: 16
   ◦ Files matching .semgrepignore patterns: 20
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 302 rules on 2096 files: 86 findings.

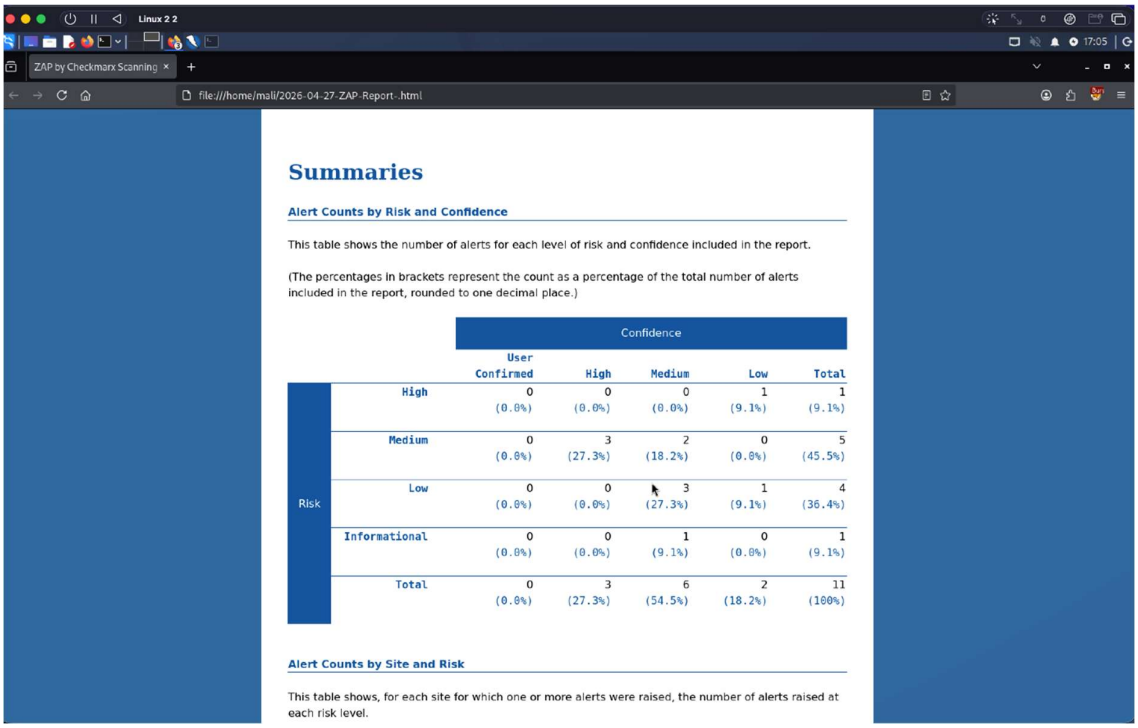
📢 Too many findings? Try Semgrep Pro for more powerful queries and less noise.
   See https://sg.run/false-positives.
```

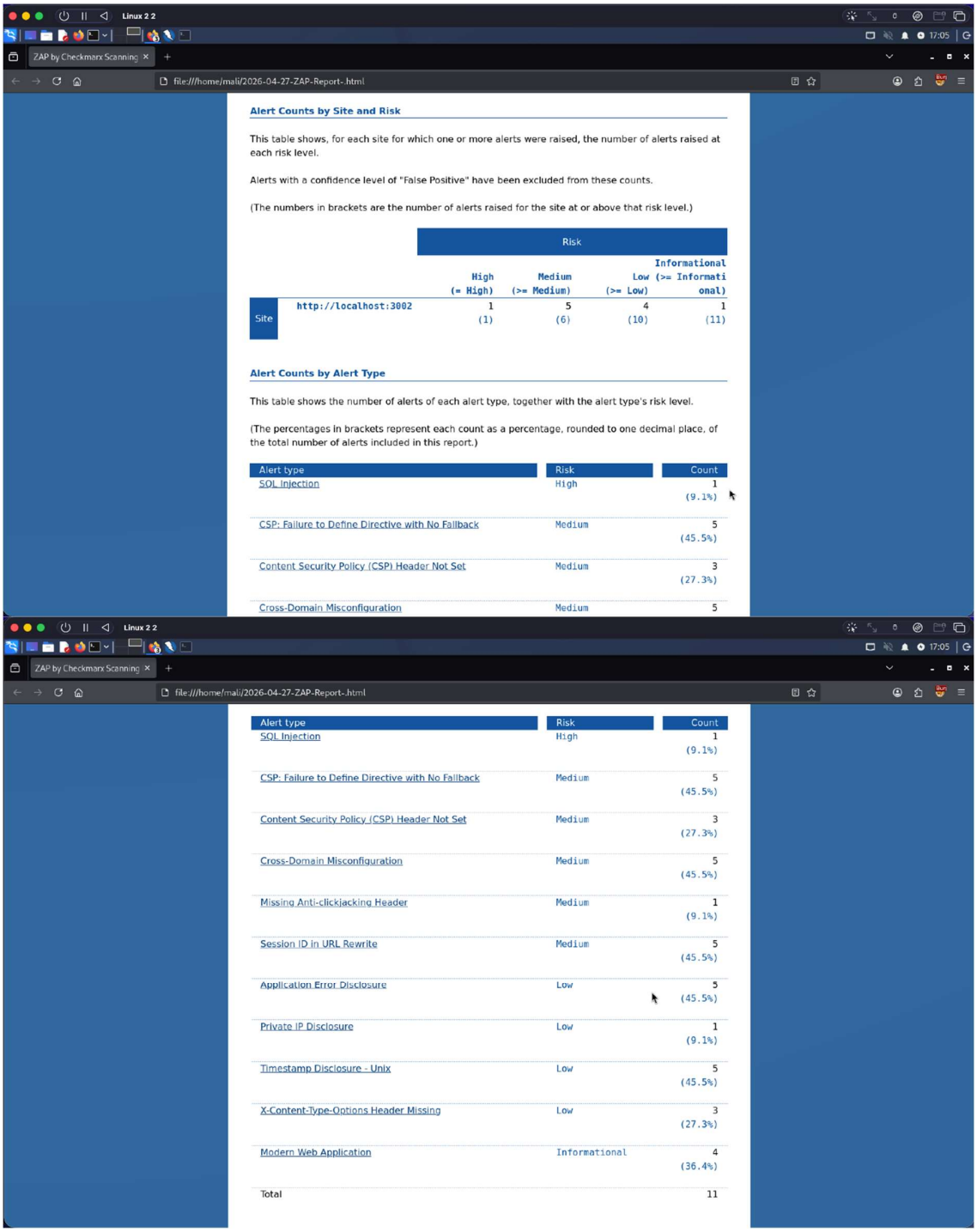
Appendix B: DAST Scan Report (2026-04-27-ZAP-Report.html) – Ingrid Francis






Linux 2 2
17:04
ZAP by Checkmarx Scanning
file:///home/mali/2026-04-27-ZAP-Report-.html
About This Report
Report Parameters
Contexts
No contexts were selected, so all contexts were included by default.
Sites
The following sites were included:
http://localhost:3002
(If no sites were selected, all sites were included by default.)
An included site must also be within one of the included contexts for its data to be included in the report.
Risk levels
Included: High, Medium, Low, Informational
Excluded: None
Confidence levels
Included: User Confirmed, High, Medium, Low
Excluded: User Confirmed, High, Medium, Low, False Positive
Summaries


Linux 2 2
17:05
ZAP by Checkmarx Scanning
file:///home/mali/2026-04-27-ZAP-Report-.html
Summaries
Alert Counts by Risk and Confidence
This table shows the number of alerts for each level of risk and confidence included in the report.
(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)
Confidence
User Confirmed
High
Medium
Low
Total
Risk
High 0 (0.0%) 0 (0.0%) 0 (0.0%) 1 (9.1%) 1 (9.1%)
Medium 0 (0.0%) 3 (27.3%) 2 (18.2%) 0 (0.0%) 5 (45.5%)
Low 0 (0.0%) 0 (0.0%) 3 (27.3%) 1 (9.1%) 4 (36.4%)
Informational 0 (0.0%) 0 (0.0%) 1 (9.1%) 0 (0.0%) 1 (9.1%)
Total 0 (0.0%) 3 (27.3%) 6 (54.5%) 2 (18.2%) 11 (100%)
Alert Counts by Site and Risk
This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.


Linux 2 2
ZAP by Checkmarx Scanning
file:///home/mali/2026-04-27-ZAP-Report-.html
Alert Counts by Site and Risk
This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.
Alerts with a confidence level of "False Positive" have been excluded from these counts.
(The numbers in brackets are the number of alerts raised for the site at or above that risk level.)
Risk
High (= High)
Medium (>= Medium)
Low (>= Low)
Informational (>= Informational)
Site
http://localhost:3002
1 (1)
5 (6)
4 (10)
1 (11)
Alert Counts by Alert Type
This table shows the number of alerts of each alert type, together with the alert type's risk level.
(The percentages in brackets represent each count as a percentage, rounded to one decimal place, of the total number of alerts included in this report.)
Alert type
Risk
Count
SQL Injection
High
1 (9.1%)
CSP: Failure to Define Directive with No Fallback
Medium
5 (45.5%)
Content Security Policy (CSP) Header Not Set
Medium
3 (27.3%)
Cross-Domain Misconfiguration
Medium
5
Linux 2 2
ZAP by Checkmarx Scanning
file:///home/mali/2026-04-27-ZAP-Report-.html
Alert type
Risk
Count
SQL Injection
High
1 (9.1%)
CSP: Failure to Define Directive with No Fallback
Medium
5 (45.5%)
Content Security Policy (CSP) Header Not Set
Medium
3 (27.3%)
Cross-Domain Misconfiguration
Medium
5 (45.5%)
Missing Anti-clickjacking Header
Medium
1 (9.1%)
Session ID in URL Rewrite
Medium
5 (45.5%)
Application Error Disclosure
Low
5 (45.5%)
Private IP Disclosure
Low
1 (9.1%)
Timestamp Disclosure - Unix
Low
5 (45.5%)
X-Content-Type-Options Header Missing
Low
3 (27.3%)
Modern Web Application
Informational
4 (36.4%)
Total
11

Appendix C: Additional Vulnerability Analyses – Ingrid Francis

The following vulnerability analyses are provided as additional supporting work completed during the project. These are not part of the required two vulnerability analyses per student and are included for completeness and to demonstrate further investigation.

Vulnerability Analysis 5: Login Admin (Injection)

a. Vulnerability description and root cause This is an **Injection** vulnerability, specifically SQL Injection (OWASP Top 10 2021: A03:2021, CWE-89). The login form does not properly validate or escape the input in the email field, so malicious SQL code can be injected and change the login query.

b. Rationale or investigative process I went to the login page and tested the email field with special characters. I entered a classic SQL injection payload like ' OR 1=1 -- and left the password field with anything. The application logged me in as the administrator. This happened because the backend query was built by directly combining user input with SQL code instead of using safe methods.

c. How the exploit works By putting the injection string in the email field, the SQL query became true for every user, allowing login as the first (admin) account. I captured the request in Burp Suite or the browser to show the modified input. Evidence includes screenshots of the login page with the payload and the successful admin login.

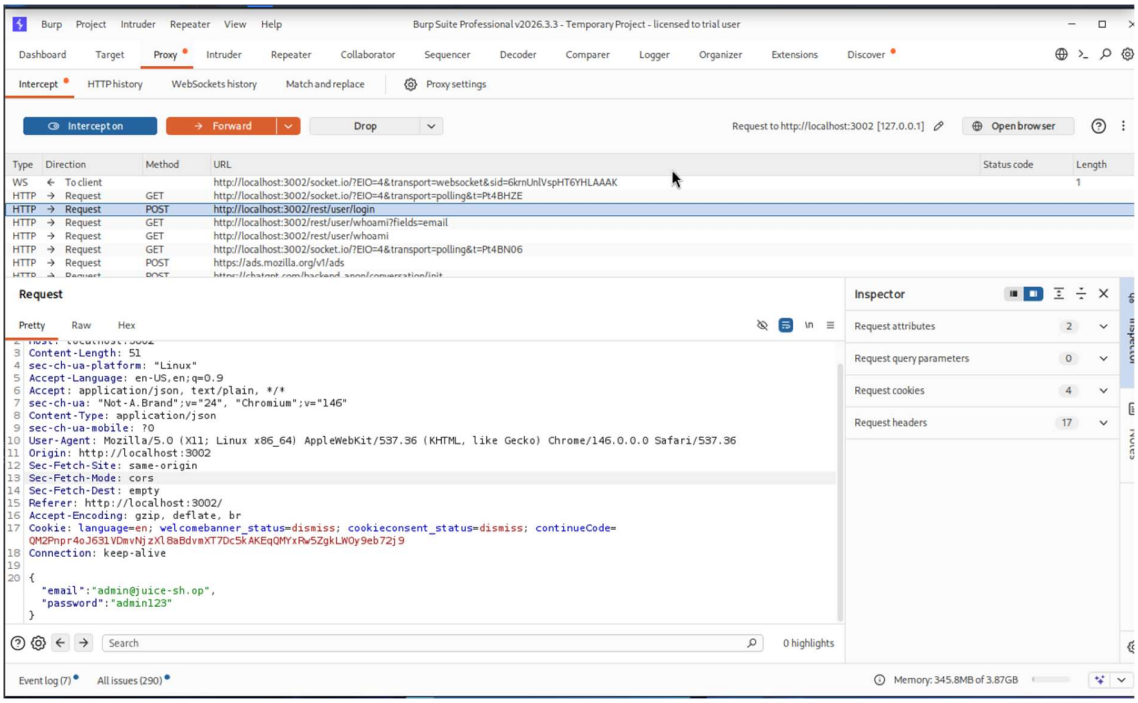


Figure 28 above: Burp Suite intercepting the login request that shows the admin credentials.

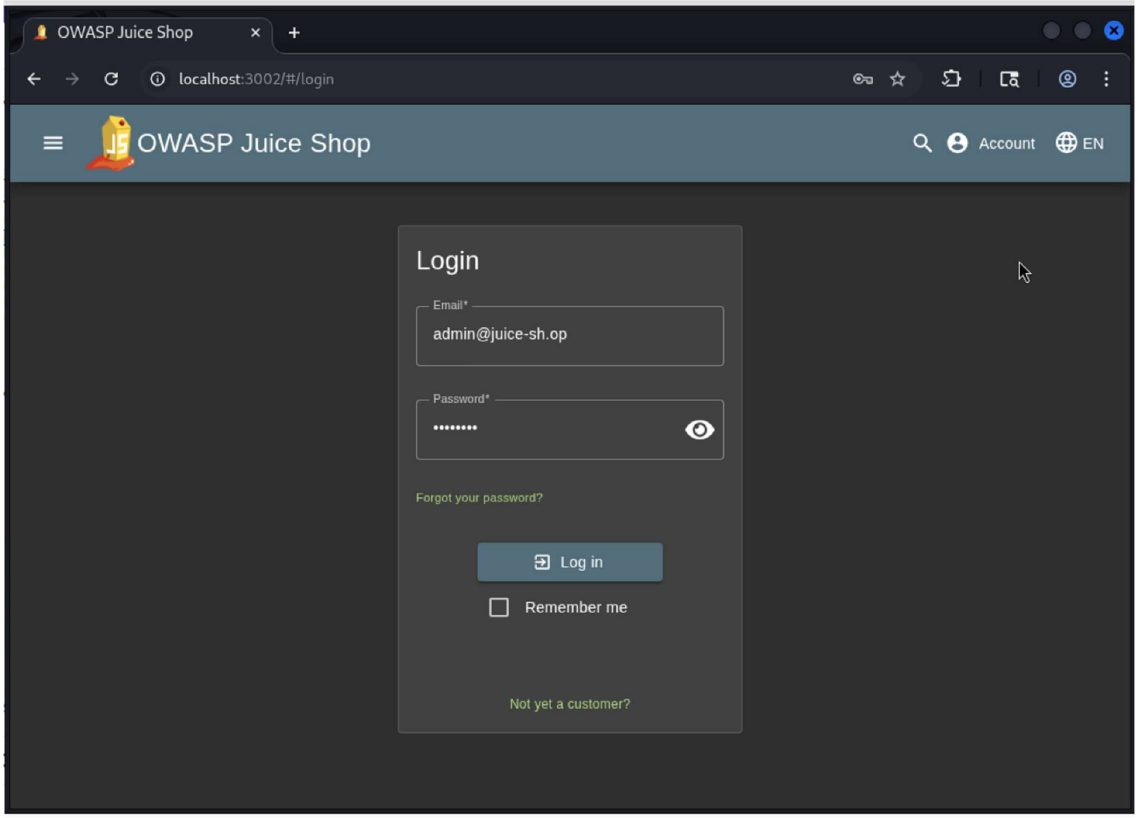


Figure 29 above: Login page.

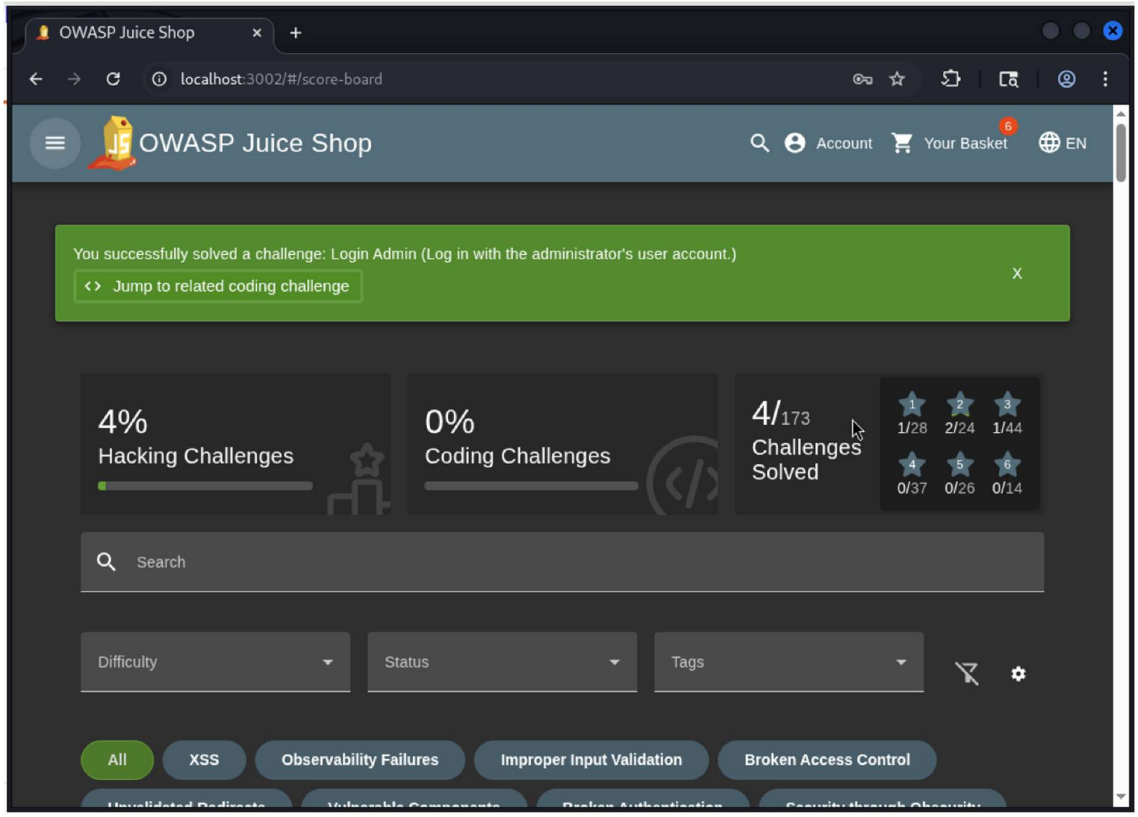


Figure 30 above: Login Admin challenge completion.

d. Potential impact An attacker could log in as any user, including the administrator, without knowing the real password. This gives full control over the application, customer data, and admin functions, which could lead to complete system compromise.

e. Recommended mitigation Use **parameterized queries** or prepared statements so that user input is treated as data, not as part of the SQL code. Never build SQL queries by string concatenation. Input validation and escaping should also be applied. This is a core secure coding practice recommended by OWASP.

Vulnerability Analysis 6: Email Leak (Sensitive Data Exposure)

a. Vulnerability description and root cause This is a **Sensitive Data Exposure** vulnerability (OWASP Top 10 2021: A02:2021, CWE-200: Exposure of Sensitive Information to an Unauthorized Actor). The /rest/user API endpoint returns user information (including emails) without requiring proper authentication or authorization.

b. Rationale or investigative process I noticed API endpoints in the network tab while using the application. I tried accessing /rest/user directly from the browser or another tool without being fully logged in. The endpoint returned a list of user data, including email addresses, even when accessed cross-domain or without valid credentials. This showed that sensitive information was not protected properly.

c. How the exploit works By calling the /rest/user endpoint directly (for example, via browser or curl), I could retrieve user emails and other details without correct authentication. Screenshots of the request and the leaked data serve as evidence.

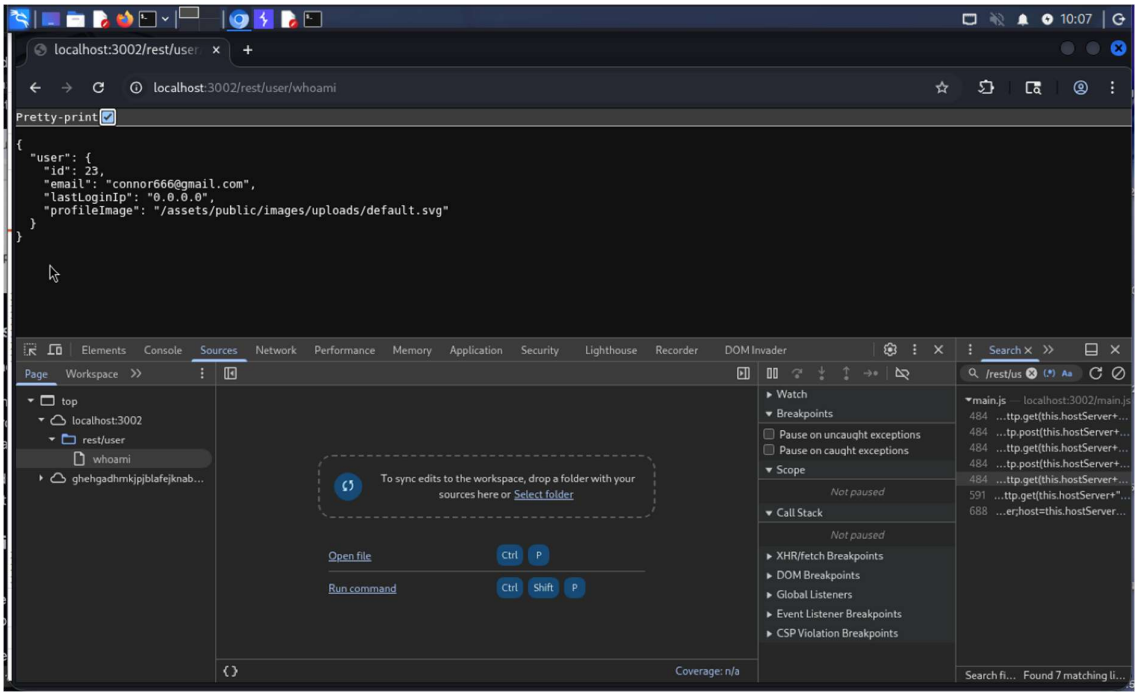


Figure 31 above: /rest/user/whoami endpoint returning user data including email address without authentication.

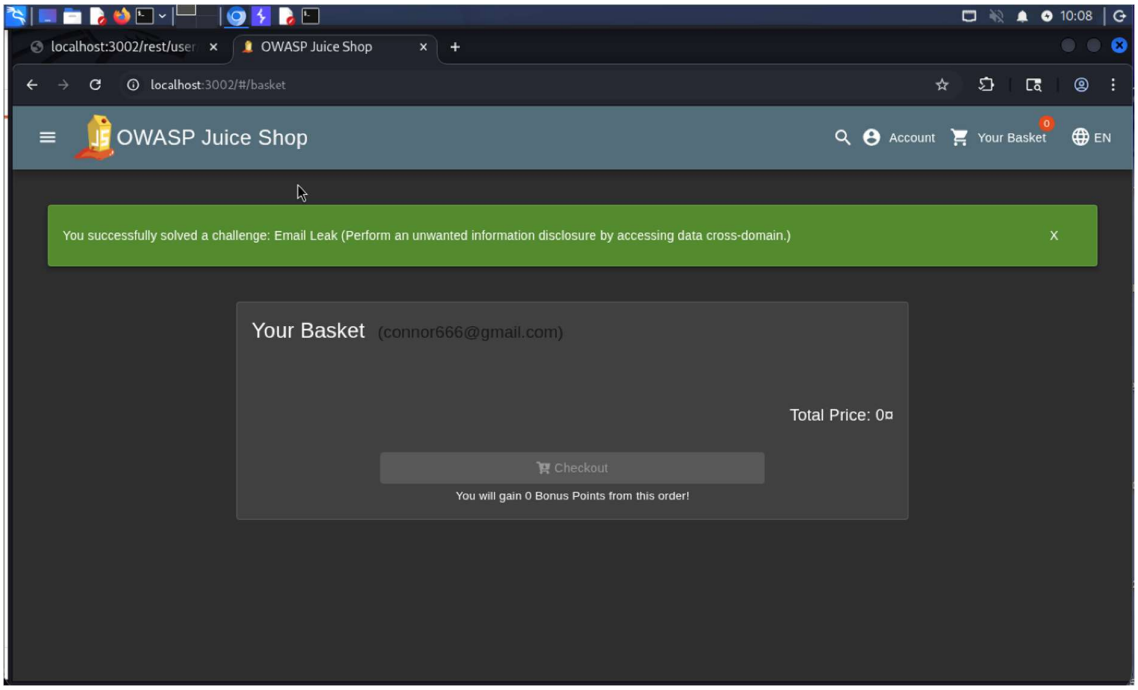


Figure 32 above: Email Leak challenge completed.

d. Potential impact Leaking email addresses and other user data can lead to spam, phishing attacks, or identity theft. In a real company, this would violate privacy laws and damage customer trust.

e. Recommended mitigation All API endpoints that return sensitive data must require strong authentication and proper authorization checks. Sensitive information should never be exposed without verification. Use techniques like data minimization and follow OWASP ASVS for API security.

Vulnerability Analysis 7: CAPTCHA Bypass (Broken Anti Automation) – Optional / Short Version

a. Vulnerability description and root cause This falls under **Broken Anti Automation** or Security Misconfiguration. The CAPTCHA protection on the feedback form can be bypassed by sending many requests quickly.

b. Rationale or investigative process I intercepted a valid feedback submission request using Burp Suite and sent it to Repeater. By resubmitting the request multiple times in quick succession (10+ requests within ~20 seconds), I observed that the server accepted all submissions without enforcing CAPTCHA validation or rate limiting.

c. How the exploit works By automating or quickly sending multiple feedback submissions, I bypassed the intended protection.

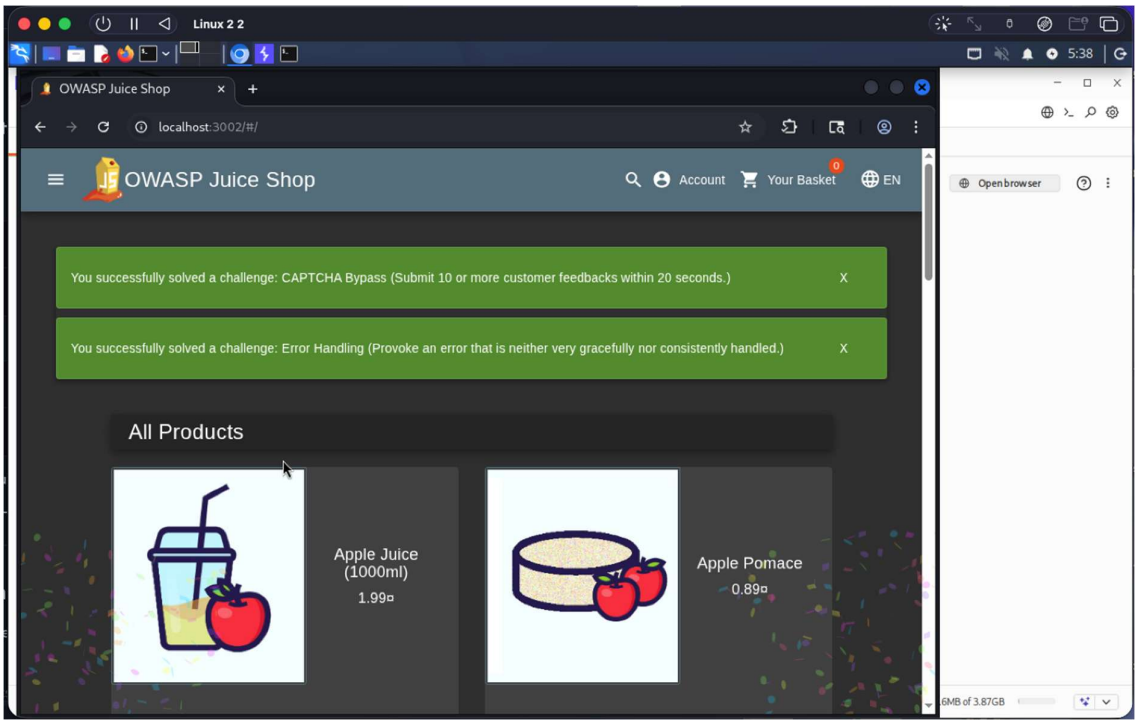


Figure 33 above: CAPTCHA challenge completed.

d. Potential impact Attackers could spam the system, flood the database, or abuse other features that should be rate-limited.

e. Recommended mitigation Implement strong rate limiting, proper CAPTCHA validation on the server side, and anti-automation measures (OWASP Proactive Controls).

Appendix D: Additional Challenges – Ingrid Francis

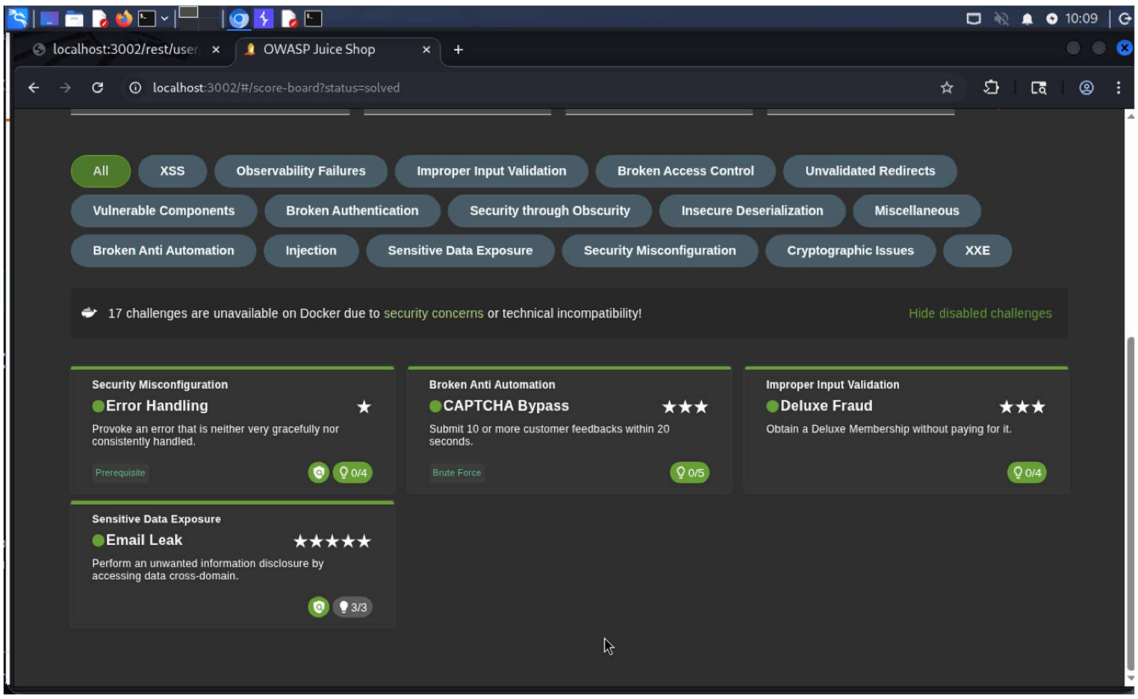


Figure 34 above: additional challenges (Error Handling, CAPTCHA Bypass, Deluxe Fraud) that were completed.

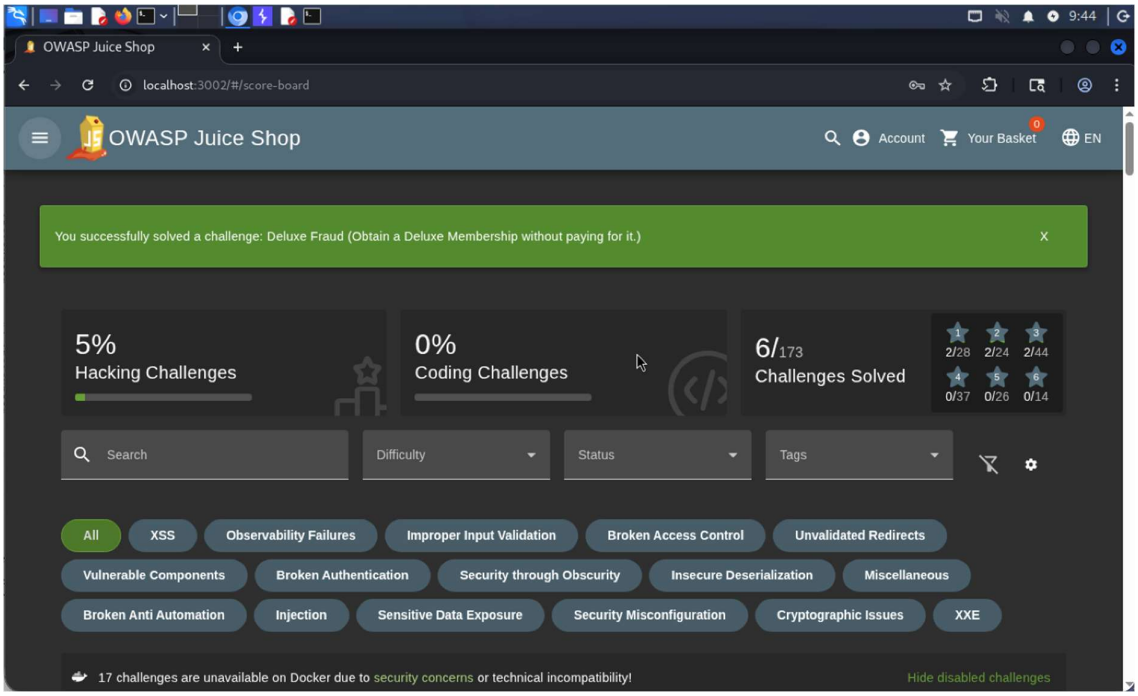


Figure 35 above: Deluxe Fraud challenge completed.

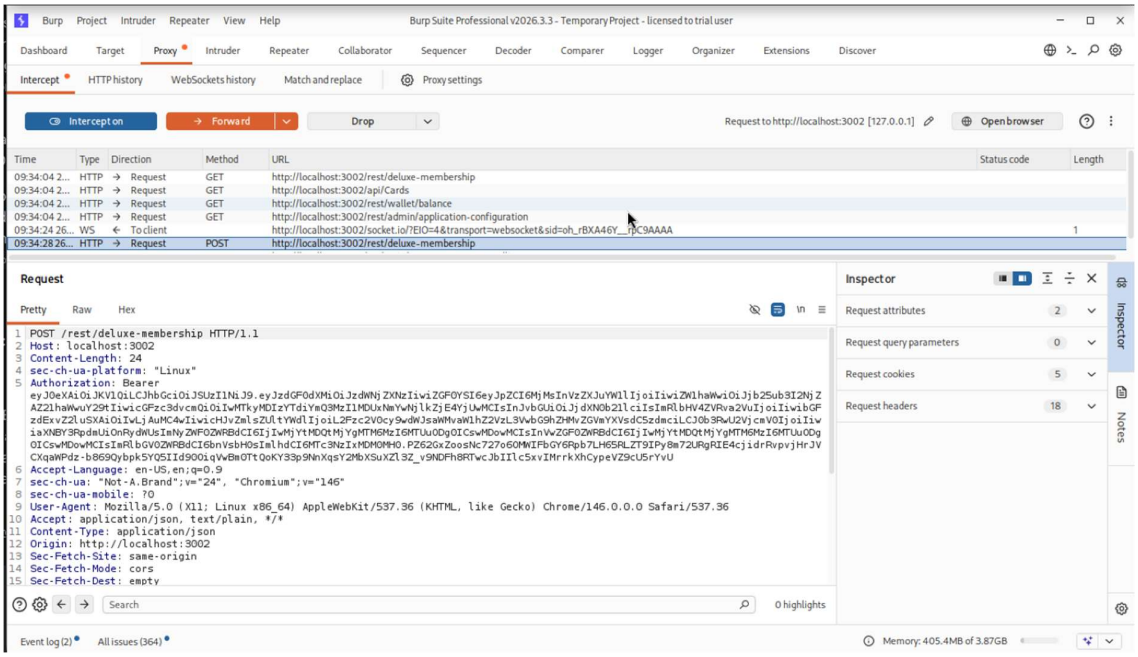


Figure 36 above: Burp Suite intercepts the deluxe membership request.

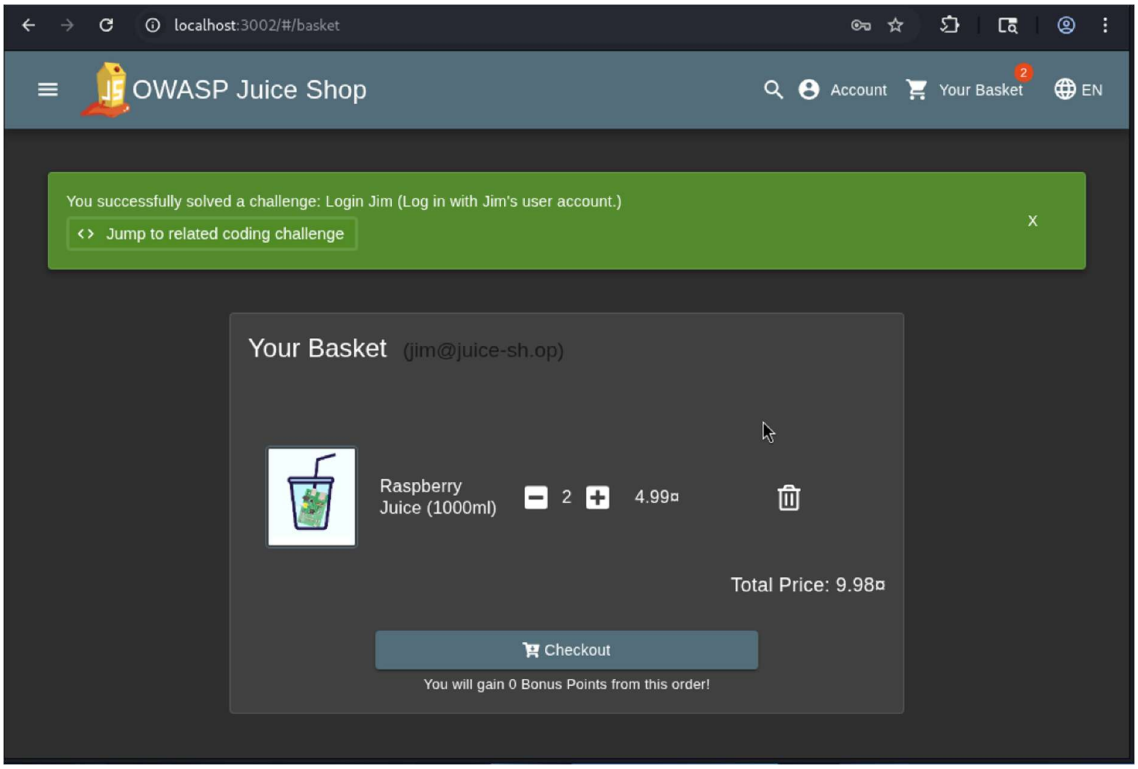


Figure 37 above: Login Jim challenge completed.

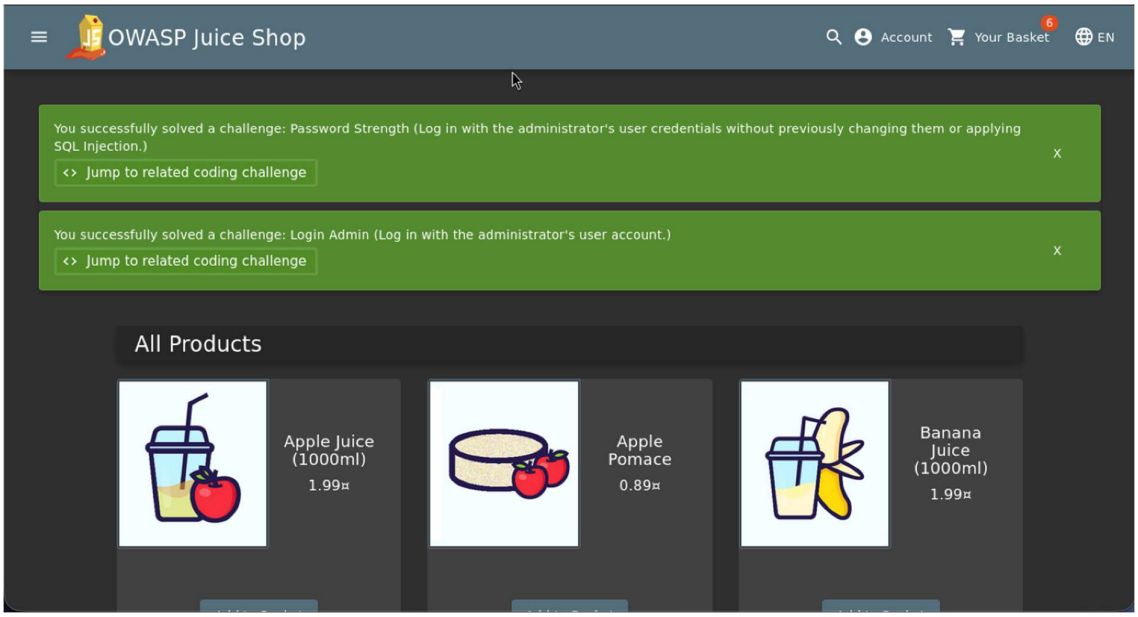


Figure 38 above: Password Strength challenge completed.